European Journal of Operational Research 307 (2023) 1–19

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Invited Review

A classification and new benchmark instances for the multi-skilled resource-constrained project scheduling problem

Jakob Snauwaert[a], Mario Vanhoucke[a,b,c,*]

[a] Faculty of Economics and Business Administration, Ghent University, Tweekerkenstraat 2, Ghent 9000, Belgium
[b] Operations and Technology Management Centre, Vlerick Business School, Reep 1, Ghent 9000, Belgium
[c] UCL School of Management, University College London, 1 Canada Square, London E14 5AA, UK

ARTICLE INFO

Article history:
Received 31 May 2021
Accepted 30 May 2022
Available online 8 June 2022

Keywords:
Project scheduling
Resource-constrained scheduling
Skills

ABSTRACT

This paper studies and analyses the multi-skilled resource-constrained project scheduling problem (MSRCPSP). We present a new classification scheme based on an existing classification scheme for project scheduling problems. This allows researchers to classify all multi-skilled project scheduling problems and its extensions. Furthermore, we propose a new data generation procedure for the MSRCPSP and introduce multiple artificial datasets for varying research purposes. The new datasets are generated based on new multi-skilled resource parameters and are compared to existing benchmark datasets in the literature. A set of 7 empirical multi-skilled project instances from software and railway construction companies are collected in order to validate the quality of the artificial datasets. Solutions are obtained through a genetic algorithm and by solving a mixed-integer linear programming formulation with CPLEX 12.6. The hardness of the multi-skilled project instances is investigated in the computational experiments. An experimental analysis studies the impact of skill availability, workforce size and multi-skilling on the makespan of the project.

© 2022 The Author(s). Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license
(http://creativecommons.org/licenses/by-nc-nd/4.0/)

1. Introduction

For many decades, research on resource-constrained project scheduling has been prevalent in academic literature. Most notably, the resource-constrained project scheduling problem (RCPSP) has been studied extensively, leading to an assembly of papers that propose various optimisation procedures and insightful contributions. The objective of the RCPSP is to minimise a cost or makespan objective subject to activity precedence relations and scarce resources. More specifically, tasks are appointed a start and finish time and resources are assigned to these tasks in order to create a feasible project schedule that adheres to the precedence relations and the limited resource capacity. There exist many variants of the RCPSP which have received increased consideration in the last few years (Hartmann & Briskorn, 2010; 2021).

This paper studies a problem that has been recurrent in recent project scheduling literature, namely the RCPSP with multi-skilled resources. Since the *multi-skilled resource-constrained project scheduling problem*, abbreviated as MSRCPSP, is a generalisation of the RCPSP, we know that it is NP-hard (Blazewicz, Lenstra, & Kan, 1983). In the MSRCPSP, activities require skills to be executed and the multi-skilled resources possess these skills. In this problem, the projects, most commonly, deal with human resources. Each individual resource masters a particular set of skills, which can be applied to the activities that require a mastered skill. Consequently, multi-skilled resources can not be assigned to all activities of the project. The precedence relations of the activities are not affected by the introduction of the skills. Furthermore, multi-skilled resources have been utilised in a multitude of different sectors, such as the IT sector (Chen, Liang, Gu, & Leung, 2017; Chen, Liang, Gu, & Zhao, 2020), research and development (Certa, Enea, Galante, & Manuela La Fata, 2009), service organisations (Valls, Pérez, & Quintanilla, 2009), telecommunications (Fırat & Hurkens, 2012), the construction sector (Liu & Wang, 2012), consulting (Correia, Lampreia-Lourenço, & Saldanha-da Gama, 2012), health care (Othman, Hammadi, & Quilliot, 2015) and nuclear research facilities (Polo-Mejía, Artigues, Lopez, & Basini, 2019).

This paper is organised as follows. In Section 2, we provide an elaborate description of the problem, together with an

* Corresponding author at: Faculty of Economics and Business Administration, Ghent University, Tweekerkenstraat 2, 9000 Ghent, Belgium.
E-mail addresses: jakob.snauwaert@ugent.be (J. Snauwaert), mario.vanhoucke@ugent.be (M. Vanhoucke).

https://doi.org/10.1016/j.ejor.2022.05.049
0377-2217/© 2022 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license
(http://creativecommons.org/licenses/by-nc-nd/4.0/)

example and a mixed-integer linear programming formulation. A classification scheme for multi-skilled project scheduling problems and extensions is given in Section 3. Related literature is presented in Section 4. Section 5 describes a data generation procedure for the MSRCPSP. The computational experiments and some managerial insights are discussed in Section 6.

2. Problem description

The multi-skilled resource-constrained project scheduling problem (MSRCPSP) can be represented by a topologically ordered acyclic activity-on-the-node network $G = (N, A)$, where N $(= \{1, \ldots, |N|\})$ is the set of nodes in the graph that stand for the activities in the project. In total, $|N|$ activities are considered, consisting of the dummy start node, end node and $|N| - 2$ project activities. The dummy nodes have a duration of zero and no resource or skill usage. Each activity i has a processing time p_i that is predefined and can not be pre-empted. A is the set of arcs that represent the finish-start precedence relations between the activities in the project with a time-lag of zero. The renewable multi-skilled project workforce that will execute the activities is specified by the resource set R, with $R = \{1, \ldots, |R|\}$. The skills of the multi-skilled resources are defined by the skill set S, with $S = \{1, \ldots, |S|\}$. We assume in this problem that the workforce consists of homogeneous resources, meaning that the efficiency is equal for all resources. However, the mastered skills of the resources, also called the categorical skills, can vary across the workforce. The categorical skills are represented by the skill distribution $b_{j,k}$, which is equal to 1 if resource k masters skill j and 0 otherwise. The number of skills that a resource k masters $(= \sum_j b_{j,k})$ is called the breadth of the resource. To create a feasible baseline schedule with start times s_i and finish times f_i, each activity $i \in N \setminus \{1, |N|\}$ requires $r_{i,j}$ resources for one or more skills $j \in S$ to be executed. These requirements are called the skill requirements. Each resource $k \in R$ can only perform one skill per activity i and is disjunctive, meaning that a resource k is not able to work on two activities at the same time. The objective is to minimize the makespan, calculated by the finish time of the final activity. All the data in the MSRCPSP studied in this paper is assumed to be deterministic, and hence, no stochastic features have been incorporated in our models.

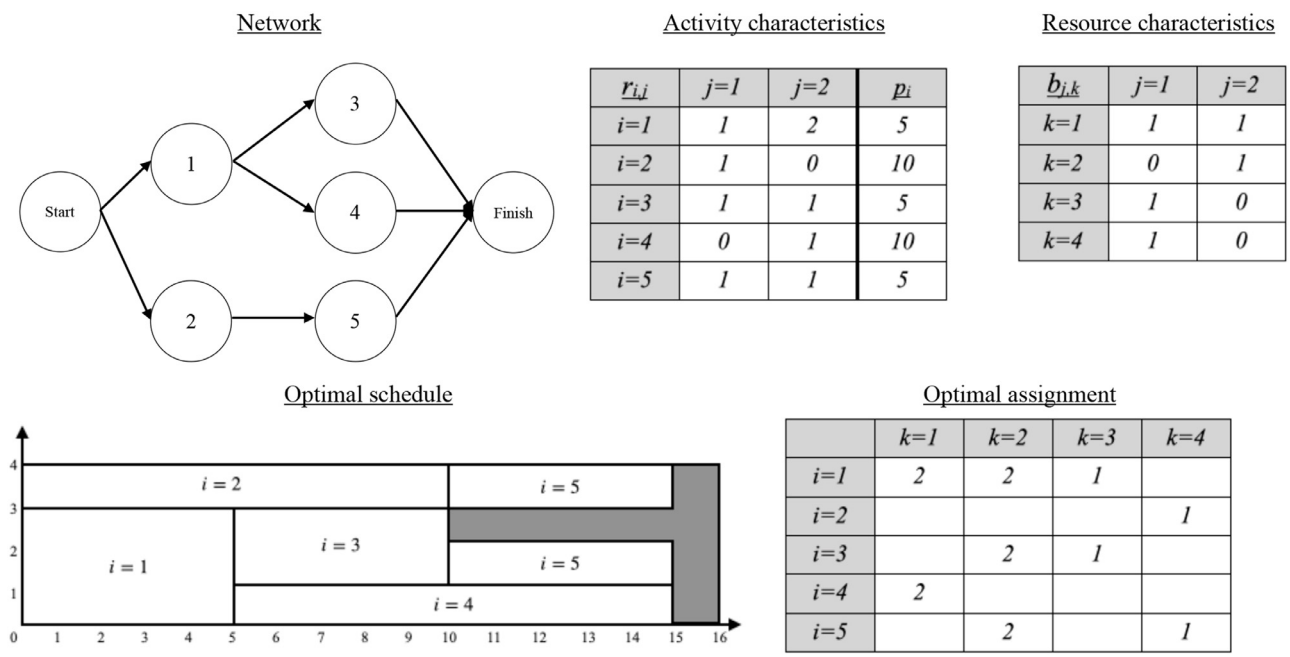
2.1. Example of the MSRCPSP

Fig. 1 displays a complete example of the MSRCPSP. The activities and their precedence relations are shown in the network on the top left. The activity characteristics show the skill requirements for both skills and the processing times of all 5 activities. For example, activity 1 requires 1 resource with skill 1 and 2 resources with skill 2. The resource characteristics show how the skills are distributed over the 4 resources. Resource 1 is multi-skilled, while the other 3 resources each master 1 skill. Below that, the optimal schedule and an optimal assignment are presented. We start the project off by scheduling activity 1 and 2 in parallel. Resources 1 and 2 perform skill 2 and resource 3 performs skill 1 in activity 1. Resource 4 performs skill 1 in activity 2. Important to realise is that if resource 1 would be assigned to activity 2, then activity 1 and 2 can not be scheduled in parallel because the required skills are not available. Once activity 1 finishes at $t = 5$, activity 3 and 4 can be started in parallel. Resources 2 and 3 are assigned to activity 3 and resource 1 to activity 4. At $t = 10$ activity 2 and 3 are finished and activity 5 is started by resources 2 and 4. The project is finalised when activities 4 and 5 finish at $t = 15$.

2.2. MSRCPSP vs. RCPSP

As can be seen from the example, there are quite a few differences between the RCPSP and the MSRCPSP. The most prominent difference is that the requirement for resources is now expressed as a requirement for skills. Hence, in the MSRCPSP, there is an indirect demand for resources that is conveyed through skills. This becomes even more clear when we consider two special cases of the MSRCPSP. If the number of mastered skills for all resources, or breadth, is equal to 1 $(\sum_j b_{j,k} = 1)$, the MSRCPSP is equivalent to the RCPSP with $|S|$ resource types. Similarly, if all resources $k \in R$ master all skills $(\sum_j b_{j,k} = |S|)$, the MSRCPSP is reduced to the RCPSP. In these two special cases of the MSRCPSP, the indirect demand of resources through skills can be ignored, and therefore they can be represented as the RCPSP. The resource demand in the RCPSP can be represented by $r_{i,k}$, which is the number of resources that are required of type k by activity i. An activity i in the MSRCPSP requires $r_{i,j}$ resources which master skill j. If and only if the resource k masters the required skill j can it work on the activity i. Since multi-skilled resources can master multiple required skills, we have to define multi-skilled constraints that specify that a resource k can only perform 1 skill j in an activity i, even if he masters multiple required skills that can be utilised in activity i. The RCPSP deals with resource scarceness, while the MSRCPSP deals with both resource scarceness and skill scarceness. This stems from the fact that an activity in the MSRCPSP requires skills as well as resources. On the one hand, there might be enough resources to perform the activity, however they do not master the right set of skills. In this case, the skill scarceness causes an infeasibility. On the other hand, the multi-skilled workforce might have the required set of skills to perform the activity, however there are not enough resources in the workforce. This can be due to the fact that 1 resource masters 2 skills that are required to feasibly execute the activity. Therefore, the resource scarceness causes an infeasibility. The example in Fig. 2 below displays that if there are enough resources available for the total requirement $(\sum_j r_{i,j})$ of an activity i, it does not imply that the activity can be finished. Fig. 2 shows the skill requirements $r_{i,j}$ and skill distribution $b_{j,k}$ of an infeasible instance. Activity 1 requires 1 resource for every skill $j \in \{1, 2, 3\}$ and activity 2 only requires skill 3. From the $b_{j,k}$-matrix, we can see that resource 1 is multi-skilled and masters skills 1 and 2. Resources 2 and 3 are both single-skilled and master skill 3. This instance is infeasible because it is not possible to execute every activity i while ensuring that every resource k only performs one skill per activity i. In this example, activity 1 can not be executed since skill 1 and 2 are mastered by the same resource, i.e. resource 1.

Each resource k possesses an integer amount of categorical skills from set S. The categorical skills of the resources are assumed to be static and do not change over time. The more skills the resource k masters, the more flexible it is. The flexibility of a resource is measured by its number of assignment possibilities to activities in the project. Usually, there are three sets of resources in a multi-skilled workforce. The first set are the single-skilled resources R_s, which only master a single skill $(\sum_j b_{j,k} = 1)$. These are the least flexible resources. Second, the completely-skilled resources R_c can perform all skills in the set S $(\sum_j b_{j,k} = |S|)$. These resources can be assigned to all skills and all activities in the project. Finally, the multi-skilled resources R_m are the resources that master from 2 up to $|S| - 1$ skills $(\sum_j b_{j,k} \in \{2, \ldots, |S| - 1\})$. The union of these 3 sets leads to the set of resources, $R_s \cup R_m \cup R_c = R$. Importantly, the value of a resource k in a project does not necessarily coincide with the number of skills it masters. Instead, the value of a mastered skill depends on the availability of that skill in the workforce and the skill requirements in the project.



$r_{i,j}$	j=1	j=2	p_i
i=1	1	2	5
i=2	1	0	10
i=3	1	1	5
i=4	0	1	10
i=5	1	1	5

$b_{j,k}$	j=1	j=2
k=1	1	1
k=2	0	1
k=3	1	0
k=4	1	0

	k=1	k=2	k=3	k=4
i=1	2	2	1	
i=2				1
i=3		2	1	
i=4	2			
i=5		2		1

Fig. 1. Example of the MSRCPSP.

Table 1
Mathematical model details.

Parameters	$r_{i,j}$	Number of resources required with skill j by activity i
	$Prec_{i,i'}$	1, if activity i precedes activity i', 0, otherwise
	p_i	Standard processing time of activity i
	$b_{j,k}$	1, if resource k masters skill j, 0, otherwise
Sets	N	Set of activities
	N^*	Set of project activities ($= N \setminus \{1, \lvert N\rvert\}$)
	R	Set of resources
	S	Set of skills
Decision Variables	$x_{i,j,k}$	1, if resource k is assigned to skill j of activity i, 0, otherwise
	$z_{i,i'}$	1, if activity i is finished before activity i' starts, 0, otherwise
	s_i	Start time of activity i

$r_{i,j}$	j=1	j=2	j=3
i=1	1	1	1
i=2	0	0	1

$b_{j,k}$	j=1	j=2	j=3
k=1	1	1	0
k=2	0	0	1
k=3	0	0	1

Fig. 2. Example of an infeasible instance.

2.3. Mathematical formulation (Table 1)

$$\text{Min} \quad s_{|N|} \tag{1}$$

$$\text{s.t.} \quad \sum_{k \in R} x_{i,j,k} = r_{i,j} \qquad \forall i \in N^*, j \in S \tag{2}$$

$$s_i + p_i - M(1 - z_{i,i'})(1 - Prec_{i,i'}) \leq s_{i'} \qquad \forall i, i' \in N \tag{3}$$

$$\sum_{j \in S} x_{i,j,k} \leq 1 \qquad \forall i \in N^*, k \in R \tag{4}$$

$$z_{i,i'} + z_{i',i} \leq 1 \qquad \forall i, i' \in N : i < i' \tag{5}$$

$$\sum_{j \in S} x_{i,j,k} + \sum_{j \in S} x_{i',j,k} \leq 1 + z_{i,i'} + z_{i',i} \qquad \forall i, i' \in N : i < i', k \in R \tag{6}$$

$$x_{i,j,k} \leq b_{j,k} \qquad \forall i \in N, j \in S, k \in R \tag{7}$$

$$s_i \geq 0 \qquad \forall i \in N \tag{8}$$

$$z_{i,i'} \in \{0, 1\} \qquad \forall i, i' \in N : i < i' \tag{9}$$

$$x_{i,j,k} \in \{0, 1\} \qquad \forall i \in N, j \in S, k \in R \tag{10}$$

The start time of the dummy end note $s_{|N|}$ is minimized in the objective function (1) in order to minimize the makespan of the project. Constraints (2) specify the skill requirements of each activity $i \in N^*$. The minimum finish-start precedence relations with a time-lag of zero are guaranteed by constraints (3). Parameter M is equal to an initial non-optimal makespan that was found through a naive method. Constraints (4) assure that each resource k can perform at most one skill per activity. The logical relation between all $z_{i,i'}$ decision variables is established in constraints (5). Constraints (6) make sure that the disjunctive character of the resources is maintained. If two activities i and i' are scheduled in parallel, then $z_{i,i'} + z_{i',i} = 0$, which means that the LHS of constraints (6) needs to be smaller than or equal to 1. As such the resource k can only perform a skill on either activity i or activity i' and will not perform

Table 2
Details and classification of multi-skilled resource-constrained project scheduling literature.

Reference	Method[a]	Dataset	Classification	Other
Hegazy et al. (2000)	Heuristic	Case Study	m,1,Flex|cpm|$C_{\max}$	Resource substitution rules
Néron (2002)	Lower bound	/	ms,1,TR|cpm|$C_{\max}$	
Bellenguez & Néron (2004)	Lower bound	/	ms,1,va,H,TR,Flex|cpm|$C_{\max}$	
Bellenguez & Néron (2007)	B&B	Own	ms,1,va,TR|cpm|$C_{\max}$	
Drezet & Billaut (2008)	Heuristic/TS	Own	ms,1,va,TR|pmtn,cpm,ρ_i, δ_i,Rvr,Svr|$L_{\max}$	Assignment changes during execution
Yannibelli & Amandi (2011)	GA	Own(ProGen)	ms,1,H,TR,E|cpm|E	
Chen & Zhang (2012)	ACO	Own(PSPLIB)	ms,1,va,H,TR,{Cost,Eff}|cpm,δ_i,Rvr,dl|C	
Correia et al. (2012)	MILP/Heuristic	Own(PSPLIB)	ms,1,TR|cpm|$C_{\max}$	
Kolisch & Heimerl (2012)	GA-TS	Own	ms,1,H,TR,Eff|gpr,cont,Rvr,dl|C	Multi-project, External resources
Yannibelli & Amandi (2013)	SA-GA	Own(PSPLIB)	ms,1,H,TR,E|cpm|$C_{\max}$,E	
Montoya et al. (2014)	B&P	Own(PSPLIB,...)	ms,1,TR|cpm|$C_{\max}$	
Myszkowski et al. (2015)	ACO	iMOPSE	ms,1,H,TR,Flex|cpm,1|$C_{\max}$,C	
Maghsoudlou et al. (2016)	IWO/NSGA-II/PSO	Own	ms,1,TR|cpm,mu|$C_{\max}$,C,Q	
Javanmard et al. (2017)	MIP/GA/PSO	Own	ms,1,H,TR,{Cost,Eff}|pmtn,cpm,δ_n|C	Level switching
Maghsoudlou et al. (2017)	Cuckoo Search	Own(RanGen)	ms,1,H,TR,{Cost,Q}|cpm|C,Q	
Almeida et al. (2019)	MILP	Correia et al. (2012)	ms,1,TR|cpm|$C_{\max}$	
Maghsoudlou et al. (2019)	MILP/ACO	Own(ProGen)	ms,1,{TR, Cost}|pmtn,cpm,δ_i|C	
Najafzad et al. (2019)	DM	Own	ms,1,H,TR,{Cost,R}|cpm,Rvr,dl|$C_{\max}$,C	Skill level switching
Polo-Mejía et al. (2019)	MILP/CP	Own	ms,1,va,TR|pmtn,cpm,δ_i|$C_{\max}$	
Zabihi et al. (2019)	TLBO	iMOPSE/Own	ms,1,H,TR,{Cost,Eff},LF|cpm|$C_{\max}$,C,E	
Chen et al. (2020)	ACO	Case Study	ms,1,H,TR,Eff,LF|cpm,1,cont|$C_{\max}$,Q,E	Project portfolio management
Hematian et al. (2020)	MILP	Own	ms,1,H,TR,{Cost,Eff,Flex},|cpm,ρ_i|C	Multi-project, under- & overcoverage
Dang Quoc et al. (2020)	Cuckoo Search	iMOPSE	ms,1,H,TR,{Flex,Eff}|cpm,1|$C_{\max}$	
Snauwaert & Vanhoucke (2021)	GA-LS	Own(RanGen)	ms,1,H,TR,Eff|cpm, *cont*|$C_{\max}$	
Zhu et al. (2021)	GP-HH	iMOPSE	ms,1,H,TR,Flex|cpm,1|$C_{\max}$,C	

[a] B&B = branch-and-bound; GA = genetic algorithm; TS = tabu search algorithm; ACO = ant colony optimisation; MI(L)P = mixed-integer (linear) programming; SA = simulated annealing; B&P = branch-and-price; IWO = invasive weeds optimisation; NSGA-II = non-dominated sorting genetic algorithm; PSO = particle swarm optimisation; DM = decision making methods; CP = constraint programming; TLBO = teaching-learning-based optimisation; LS = local search; GP-HH = genetic programming hyper-heuristic.

two activities in parallel. The categorical skill constraints (7) specify that a resource k can only perform a mastered skill j, with $b_{j,k} = 1$. Finally, constraints (8)–(10) display the domain of the decision variables.

3. Classification

In this section, we present a classification scheme that allows the representation of multi-skilled resource-constrained project scheduling problems and their extensions. Scouting the literature for multi-skilled project scheduling problems we found that there is an immense variety in research regarding these problems. On top of this, many of the researched problems tend to propose solution procedures for problems with characteristics that were previously unconsidered. This is mainly due to the variable nature of multi-skilled resources. More specifically, the introduction of individual resources that can each impact the project in a different manner generates a great amount of extensions and research opportunities. For these reasons, an accurate and rigorous notation is necessary that can represent all project scheduling problems that deal with multi-skilled resources. The classification scheme proposed in this section establishes a systemic method to classify these problems. The purpose of this classification scheme is to make it more accessible and straightforward for authors to highlight (and for readers to identify) the characteristics of the project scheduling problem at hand. Especially since project scheduling problems with multi-skilled resources regularly have many characteristics which always need to be spelled out in great detail in current research papers. With the ever-expanding research on the MSRCPSP, this classification scheme promotes easier verification of which parts of the literature are left unconsidered and unstudied. It also allows for researchers to situate their new problems into the current literature. Finally, it facilitates comparing the different multi-skilled problems and their solution procedures. We demonstrate the potential of this new multi-skilled classification scheme by applying it to all papers in our literature study in Section 4 (see Table 2). The specifics of the classification scheme, that extends the classification scheme of Herroelen, Demeulemeester, & De Reyck (1999), are presented in Appendix A.

4. Related literature

MSRCPSP: Hegazy, Shabeeb, Elbeltagi, & Cheema (2000) were the first to propose a multi-skilled project scheduling problem. In order to include resources that can deal with multiple skills, their paper implemented resource substitution rules. These rules allowed each resource of a certain type to substitute for another type of resources, which implied that the complete workforce masters all skills. In Néron (2002), skillsets are proposed to characterise multi-skilled resources instead of resource substitution rules. This was the first paper to define the MSRCPSP how it is defined in the current paper. Néron (2002) also presents two possible lower bounds. More recently, multiple *exact approaches and heuristic algorithms* have been published for the MSRCPSP. A branch-and-bound algorithm for the MSRCPSP is presented in Bellenguez & Néron (2007) that uses a branching scheme that is based on reduction of the slack of activities. Montoya, Bellenguez-Morineau, Pinson, & Rivreau (2014) constructed a column generation algorithm within a branch-and-price procedure. Their branching strategy was based on reducing the time windows of the activities. In Drezet & Billaut (2008), a greedy heuristic and a tabu search algorithm are proposed for a multi-skilled scheduling problem with preemption, deadlines, daily legal working restrictions and a maximal lateness objective. Correia et al. (2012) propose a continuous time mathematical formulation for the MSRCPSP with binary sequencing variables and integer natural date variables for the starting times of the activities. Furthermore, they present valid inequalities to simplify the mathematical formulation and a two-step approach that combines a heuristic and a local search procedure. Maghsoudlou, Afshar-Nadjafi, & Niaki (2016) address a multi-skill multi-mode project scheduling problem with makespan, cost and quality minimisation objectives. In their paper, they compare the results of

an invasive weeds algorithm, a non-dominated sorting genetic algorithm and a particle swarm algorithm. Maghsoudlou, Afshar-Nadjafi, & Niaki (2019) created a hybrid ant colony metaheuristic with local searches for the MSRCPSP with preemption and due date intervals. The objective in their problem is to minimise the total costs, which includes preemption and due date penalty costs as well as processing costs. Polo-Mejía et al. (2019) solve a multi-skilled project scheduling problem with partial and full preemption, deadlines and variable resource availability in a nuclear research facility with multiple programming approaches. Multiple integer and mixed-integer linear programming formulations are designed for the MSRCPSP in Almeida, Correia, & Saldanha-da Gama (2019). Although the RCPSP has been addressed in a stochastic environment in a multitude of research papers (Herroelen & Leus, 2005), the research on stochastic project scheduling problems with multi-skilled resources is very limited. Alvanchi, Lee, & Abou Rizk (2012) presented a simulation-based approach that focusses on the dynamic evolution of skills during a project scheduling problem. More recently, Felberbauer, Gutjahr, & Doerner (2019) proposed two different optimization approaches for a multi-skilled project scheduling and personnel planning problem with stochastic demand.

Extensions of the MSRCPSP: Bellenguez & Néron (2004) extended the MSRCPSP by introducing *hierarchical levels of skill*. In their work, multi-skilled resources were not only differentiated by the skills they possess, but also by the level of that skill. Skill-levels had an impact on the activity requirements, i.e. an activity requires a skill with a certain level to be executed. Myszkowski, Skowroński, Olech, & Oślizło (2015) also study this extension and exhibit an ant colony optimisation procedure. In their paper they propose the iMOPSE dataset that includes instances for the MSRCPSP with hierarchical skills. Zhu, Lin, Li, & Wang (2021) constructed a genetic programming hyper-heuristic approach for the MSRCPSP with hierarchical skills that is tested on the iMOPSE dataset of Myszkowski et al. (2015). Other papers have focussed their research on the MSRCPSP with hierarchical skill levels that have a different impact on the scheduling problem than the activity requirements. In Dang Quoc, Nguyen Doan, & Xiong (2020), the hierarchical skill levels represent efficiency levels of the multi-skilled resources that have an impact on their speed of work and therefore the activity durations. For this problem, they set out a cuckoo search algorithm. Their solution approach was also tested on the iMOPSE dataset. A similar problem is researched in Snauwaert & Vanhoucke (2021), they developed a genetic algorithm with activity-based and resource-based local searches. Javanmard, Afshar-Nadjafi, & Niaki (2017) address a multi-skilled resource investment problem, in which the availability of skills is a decision variable. In their problem, there is an impact of hierarchical skills on the activity durations as well as the usage cost of the resources. Chen & Zhang (2012) and Zabihi, Kahag, Maghsoudlou, & Afshar-Nadjafi (2019), respectively, propose an ant colony optimisation and a teaching-learning based metaheuristic for an extension of the MSRCPSP where hierarchical levels of skills are used to distinguish the efficiency and the cost of the resources in the project. Yannibelli & Amandi (2011) implement a genetic algorithm to solve the MSRCPSP with the hierarchical levels of skills in order to maximise the total employee effectiveness in the final schedule of the project. In Yannibelli & Amandi (2013) a hybrid meta-heuristic approach is established for this problem. In the problem suggested in Najafzad, Davari-Ardakani, & Nemati-Lafmejani (2019), hierarchical skills are used to differentiate the consumption of a commodity and the cost of the resources. In Maghsoudlou, Afshar-Nadjafi, & Niaki (2017) hierarchical skills determine a difference in cost and rework risk of the resources. They present three different hybrid cuckoo search algorithms in order to minimise the cost and total rework risk of the activities. An extensive and essential review of the literature on scheduling problems with multi-skilled resources is addressed in Afshar-Nadjafi (2020). This review paper includes published articles that deal with multi-skilled resources in project scheduling, staff scheduling, production scheduling, workforce composition and other disciplines.

Multi-project scheduling and project portfolio management: Multi-skilled resources have also been used in the *multi-project and project portfolio literature*. Kolisch & Heimerl (2012) present a hybrid genetic algorithm - tabu search procedure for a multi-project scheduling problem with skilled resources with variable efficiencies that influence activity durations. Chen et al. (2020) suggest a multi-objective Pareto ant colony optimisation algorithm for a problem with multi-skilled resources that are characterised by learning and forgetting in a project portfolio problem for IT product development. Hematian, Seyyed Esfahani, Mahdavi, Mahdavi-Amiri, & Rezaeian (2020) address a multi-project multi-skill project scheduling problem with learning and uncertain activity durations that are represented by fuzzy numbers.

Summary and classification: Table 2 presents a list with papers of this section that deal with the MSRCPSP or an extension to the MSRCPSP. The column "Reference" in Table 2 presents the paper reference, while column "Method" displays the proposed method to solve the problem at hand. The column "Dataset" shows the used dataset, followed by the classification of the studied multi-skilled project scheduling problem in column "Classification". The column "Other" indicates whether the study deals with some other specific extensions not explained in its classification.

Literature growth: Fig. 3 displays the recent growth pattern of research on the MSRCPSP. At the start of this century, the first peer-reviewed articles studying the MSRCPSP were published. However, the MSRCPSP only gained traction in the academic world from 2010 onwards. This can be due to the fact that the improving computational power allows researchers to study more difficult and larger problems. Furthermore, a substantial part of the new research after 2015 is a result of the iMOPSE dataset published in Myszkowski et al. (2015). Since the publication of this dataset, a myriad of meta-heuristic procedures for the MSRCPSP and its extensions were developed and compared.

Multi-skilled resources in other scheduling problems: In this paper, we study skills in a resource-constrained project scheduling problem, which is only a restricted view on the possibility of skills in scheduling problems. Apart from resource-constrained project scheduling problems, multi-skilled resources have been featured abundantly in other scheduling problems such as workforce scheduling, production scheduling and multi-skilled work assignment problems applied to various types of working sectors. An important overview paper was published recently for workforce scheduling problems with multi-skilled resources. De Bruecker, Van den Bergh, Beliën, & Demeulemeester (2015) present a literature review with both technical and managerial papers of workforce planning problems that regarded resources with skills. Among other things, they set out the determinants and consequences of skills in the literature. Cai & Li (2000) introduce a genetic algorithm for a staff scheduling problem with cost and staff surplus objectives for a cargo forwarding case. Avramidis, Chan, Gendreau, L'ecuyer, & Pisacane (2010) study a multi-skilled staffing and scheduling problem with agents in a call center, for which they created an approach based on simulation techniques. Dynamic multi-agent systems were used in Othman et al. (2016) to deal with a multi-skilled task scheduling problem in a health organisation with stochastic patient arrivals. De Bruecker, Beliën, Van den Bergh, & Demeulemeester (2018) constructed a three-stage mixed integer programming approach for a team-composition and staff scheduling problem with working and training costs at an aircraft maintenance company.

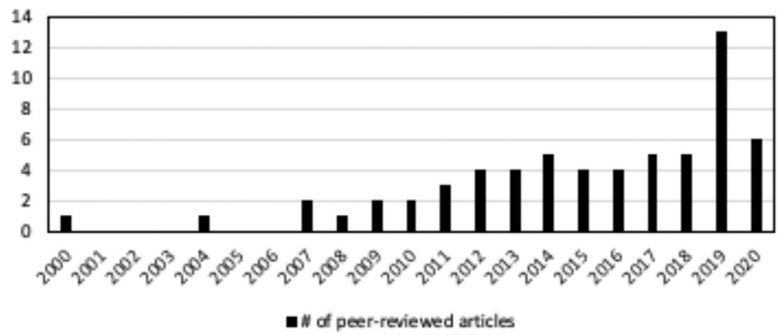


Fig. 3. Historic number of peer-reviewed articles on MSRCPSP.

5. Data generation

In this chapter, we will present the various elements of the data generation procedure. The multi-skilled project metrics that are used as input values for the procedure are explained in Section 5.1. In Section 5.2, we set out the new data creation procedure. The specifics of the dataset design are described in Section 5.3. Finally, we evaluate and validate the different multi-skilled datasets that are made available in Section 5.4.

5.1. Multi-skilled project metrics

Skill factor: The skill factor SF_i of an activity i represents how many of the $|S|$ skills will be required in activity i. It is presented in Correia et al. (2012) as follows:

$$SF_i = \frac{\sum_j y_{i,j}}{|S|} \tag{11}$$

The binary variable $y_{i,j}$ is equal to 1 if activity i requires skill j and 0 otherwise.

$$y_{i,j} = \begin{cases} 1, & \text{if } r_{i,j} > 0. \\ 0, & \text{otherwise.} \end{cases} \tag{12}$$

Project skill factor: The project skill factor SF specifies the ratio of the average number of skills $|S|$ required in each activity. This multi-skilled project parameter defines the density of the skill requirements matrix.

$$SF = \frac{\sum_i \sum_j y_{i,j}}{|N| \cdot |S|} = \frac{\sum_i SF_i}{|N|} \tag{13}$$

Assume $|S| = 4$, then a value of $SF = 0.25$ means that exactly 1 of the 4 skills will be required in all activities. Hence, $SF_i = 0.25,\ \forall i \in N^*$. Similarly, an $SF = 1$ signifies that all 4 skills will be required by all activities. So all skill requirements will be strictly greater than 0, $r_{i,j} > 0$, and $SF_i = 1,\ \forall i \in N^*$. For $SF = 0.5$ and $SF = 0.75$, on average each activity will require, respectively, 2 and 3 skills in each activity. However, all activities can still require from 1 up to $|S|$ skills. Therefore, $SF_i \in [0.25; 1],\ \forall i \in N^*$.

Skill strength: The skill strength SS_j is defined by the ratio of the amount a skill j occurs in the workforce and the amount skill j is required in the project. The skill strength is used to measure the scarceness of the skill in the project. It is equivalent to the resource strength for the multi-mode resource-constrained project scheduling problem as presented in Kolisch, Sprecher, & Drexl (1995). The skill strength is calculated as follows:

$$SS_j = \frac{a_j - a_j^{min}}{a_j^{max} - a_j^{min}} \tag{14}$$

The number of resources that master skill j is described by a_j, which is called the *skill availability* of skill j. Obviously, $a_j = \sum_k b_{j,k}$. The minimum skill availability $a_j^{\min}$ $(= \max_i(r_{i,j}))$ represents the minimum number of skills of skill j that are required to feasibly execute the complete project. The maximum skill availability $a_j^{\max}$ is determined by the sum of the skill requirements of skill j over all activities $(= \sum_i r_{i,j})$. In the denominator, we chose to use the required skills over all activities $(a_j^{\max} = \sum_i r_{i,j})$ and not the peak requirement in an earliest start schedule because the MSRCPSP deals with various other objectives apart from makespan, as can be seen from the classification scheme in Section 3. As a result, we do not omit any skill availabilities from the dataset that could lead to viable instances for other objectives than the makespan objective.

Project skill strength: The project skill strength SS measures the total skill scarceness in the project by comparing the total number of skills available in the project to the number of skills required in the project.

$$SS = \frac{\sum_j a_j - \sum_j a_j^{min}}{\sum_j a_j^{max} - \sum_j a_j^{min}} \tag{15}$$

An SS equal to 0 means that the minimum amount of skills are available in the workforce to execute the project. If $SS = 0$, then all skill strengths SS_j will be zero for all skills in the set S, which means that all skills in the project will be scarce. A project skill strength equal to 1 signifies that for each skill required in the project a skill will be available in the workforce. Therefore, $SS_j = 1, \forall j \in S$, and the availability of all skills is very high. If the value of the project skill strength SS lies between 0 and 1, the skill strength SS_j can vary across the skills. We incorporate $\frac{\alpha}{\alpha_{\max}}$ to introduce variability over the skill availabilities (Labro & Vanhoucke, 2008). Hence, we will use a skill strength variability $SS\alpha$ to analyse the impact of variable skill scarceness in the workforce. If $SS\alpha = 0$ the skill availability will be proportional for all skills. If $SS\alpha = 1$ a single skill type is mastered by all resources, while the other skill types have minimal availability. This process is further clarified in Section 5.2. The current workforce will not be able to feasibly execute the project if either the skill strength or the project skill strength are lower than zero, because in that case not enough skills are available. If an SS_j is larger than 1, then the available number of skills for skill j is larger than the total requirement, which means that there is an excess of skills that will never be used. The SS_j is assumed to be equal to 1 if the maximum skill availability $a_j^{\max}$ is equal to the minimum skill availability $a_j^{\min}$, causing the denominator to be equal to 0.

Resource availability: The resource availability RA is a metric for the resource scarceness of the instance. In addition, it quantifies how multi-skilled the resources are and how the available skills in the instance are distributed over the resources. In the RA, the total number of resources in the workforce $|R|$ is divided by the maximum number of resources possible for execution of the project $R^{\max}$, both of these values are adjusted by the minimum

$r_{i,j}$	j=1	j=2
i=1	2	0
i=2	0	2
i=3	2	0
i=4	0	2

$b_{j,k}$	j=1	j=2
k=1	1	1
k=2	1	1

SS low

RA low

$b_{j,k}$	j=1	j=2
k=1	1	0
k=2	1	0
k=3	0	1
k=4	0	1

SS low

RA high

$b_{j,k}$	j=1	j=2
k=1	1	1
k=2	1	1
k=3	1	1
k=4	1	1

SS high

RA low

$b_{j,k}$	j=1	j=2
k=1	1	0
k=2	1	0
k=3	1	0
k=4	1	0
k=5	0	1
k=6	0	1
k=7	0	1
k=8	0	1

SS high

RA high

Fig. 4. Example of various $SS - RA$ workforce combinations.

$b_{j,k}$	j=1	j=2	j=3
k=1	1	1	0
k=2	1	1	0
k=3	1	0	1
k=4	1	0	1
k=5	0	1	1
k=6	0	1	1

$SS\alpha$ low

$RA\alpha$ low

$b_{j,k}$	j=1	j=2	j=3
k=1	1	1	1
k=2	1	1	1
k=3	1	1	1
k=4	1	0	0
k=5	0	1	0
k=6	0	0	1

$SS\alpha$ low

$RA\alpha$ high

$b_{j,k}$	j=1	j=2	j=3
k=1	1	1	0
k=2	1	1	0
k=3	1	1	0
k=4	1	1	0
k=5	1	1	0
k=6	1	0	1

$SS\alpha$ high

$RA\alpha$ low

$b_{j,k}$	j=1	j=2	j=3
k=1	1	1	1
k=2	1	1	1
k=3	1	1	1
k=4	1	0	0
k=5	1	0	0
k=6	1	0	0

$SS\alpha$ high

$RA\alpha$ high

Fig. 5. Example of various $SS\alpha - RA\alpha$ workforce combinations (given $|S| = 3$, $|R| = 6$ and $\sum_j a_j = 12$).

number of resources needed to create a feasible workforce $R^{\min}$:

$$RA = \frac{|R| - R^{min}}{R^{max} - R^{min}} \tag{16}$$

The $R^{\max}(= \frac{\sum_j \sum_k b_{j,k}}{b^{\min}} = \frac{\sum_j a_j}{b^{\min}})$ is the number of resources over which all skills can be distributed if all resources mastered the minimum amount of skills $b^{\min}$. $R^{\min}$ cannot be determined in an equivalent manner as $R^{\max}$, with $b^{\max}$ instead of $b^{\min}$, because than infeasible combinations of skills $|S|$ and resources $|R|$ can occur. Therefore, we calculate $R^{\min}$ by solving a small assignment problem that takes the total number of skills $(\sum_j \sum_k b_{j,k})$ into account. If $R^{\max} = R^{\min}$, then we assume that $RA = 1$. On the one hand, if RA is equal to 0, then the instance has the minimum number of possible resources and hence the workforce is in its most multi-skilled state. On the other hand, an $RA = 1$ signifies that the workforce contains the maximum number of resources, which means that all resources will be single-skilled, $\sum_j b_{j,k} = 1, \forall k \in R$. If the RA lies between 0 and 1, the breadth of the resources can vary from single-skilled to completely-skilled. Similarly to the $SS\alpha$, the resource availability variability $RA\alpha$ is introduced by use of $\frac{\alpha}{\alpha_{\max}}$. If $RA\alpha = 0$, the breadth of all multi-skilled resources in the workforce will be the same. For an $RA\alpha$ of 1 the workforce will consist of a combination of single-skilled and completely-skilled resources. Therefore, we can now analyse workforces for which the breadth across all resources is equal and workforces for which the breadth varies from 1 to $|S|$.

In Figs. 4 and 5 we present workforces with different parameter settings to further explain the aforementioned parameters. Fig. 4 displays the skill requirements $r_{i,j}$ and various different feasible workforces that can be generated based on these skill requirements, the project skill strength SS and the resource availability RA. This example contains 4 activities, two of which require two resources that master skill 1, the other two activities require two resources that master skill 2. Hence, the workforce needs to be comprised of at least 2 resources $(= \max_i(\sum_j r_{i,j}) = R^{\min})$ and the requirements demand a skill availability of at least 2 for both skill 1 and skill 2 $(= a_j^{\min})$. If we set $SS = 0$ and $RA = 0$, we generate the most left workforce in Fig. 4. This is the smallest possible feasible workforce in terms of mastered skills and available resources. For the second workforce we increase the RA value, which means that we need to increase the number of resources given the number of mastered skills. Seeing that the latter is equal to 4, we generate a single-skilled workforce with a total of 4 resources. The third workforce is generated using a high project skill strength. In this case, the number of available skills $\sum_j a_j$ is equal to the number of required skills $\sum_i \sum_j r_{i,j} = 8$. Since the resource availability is set to 0, we generate the most multi-skilled workforce with a total of 4 resources. The last workforce on the right has the same project skill strength as the third workforce and therefore the number of mastered skills is equal to 8. However, the ressource availability is high and the number of resources is maxed out to 8 single-skilled resources. To summarise, the project skill strength SS determines the number of mastered skills bounded by the total skill requirement. An $RA = 0$ $(RA = 1)$ then creates the smallest (largest) workforce in terms of resources constrained by the number of mastered skills determined by the SS. The following

Fig. 5 sets out workforces for various values of the skill strength variability $SS\alpha$ and the resource availability variability $RA\alpha$. In this figure, it is assumed that the number of skills $|S|$ is 3 and the number of resources $|R|$ is equal to 6. Moreover, in this example there are 12 mastered skills to be divided over the resources ($= \sum_j a_j$). The workforce on the left has low skill strength variability and low resource availability variability. This shows in the fact that the skill availability is equal to 4 for all skills and all resources have a breadth $\sum_j b_{j,k}$ equal to 2. In the second workforce, we increase the variability over the resources ($RA\alpha \uparrow$). As a result, there is large disparity among the breadth of the resources. The first 3 resources have breadth equal to 3 and the latter 3 resources are single-skilled. Next, we only increase the skill strength variability for the third workforce. In this case, the breadth remains 2 for all resources, but skill 1 and 2 have a high skill availability compared to skill 3. Lastly, for the fourth workforce both variabilities are set to high. The workforce consists of completely-skilled and single-skilled resources and the availability of skill 1 is a lot higher than the availability of the other 2 skills.

5.2. Data creation procedure

In this section, we will present the multi-skilled project scheduling problem data generator. Algorithm 1 describes the outline of our data creation procedure. For a given number of activities $|N|$ and a serial-parallel network indicator value SP, a project network is obtained. The serial-parallel network indicator value SP (I_2 in Vanhoucke, Coelho, Debels, Maenhout, & Tavares (2008)) is a parameter that determines the structure of the network, ranging from 0 for a completely parallel network to 1 for a completely serial network. In total, 500 networks per $|N|$ and SP were generated by the *RanGen2* generator (Vanhoucke et al., 2008). For each setting and instance, a project network is randomly selected from this set of 500 networks. Afterwards, the processing time p_i of each activity $i \in N^*$ and the finish-start precedence relations between the activities are retrieved.

Algorithm 1 MSRCPSP instance generation.

1: *ReadNetwork*$(|N|, SP)$
2: $r_{i,j} \leftarrow$ *GenerateRequirements*$(|N|, |S|, SF, r_i^{\max}, r_{i,j}^{\max})$
3: **do**
4: $\quad a_j \leftarrow$ *GenerateSkills*$(|N|, |S|, r_{i,j}, SS, SS\alpha)$
5: $\quad b_{j,k} \leftarrow$ *GenerateResources*$(|N|, |S|, r_{i,j}, RA, RA\alpha, b^{\min}, b^{\max}, a_j)$
6: **while** FeasibilityCheck$(|N|, |S|, |R|, r_{i,j}, b_{j,k})$

`GenerateRequirements()`: Next, the skill requirements $r_{i,j}$ are determined in Algorithm 2. A value for the variation in the skill factor along all activities in N^* is randomly generated, represented by $SF\alpha \in \{0, 0.5, 1\}$. Based on the project skill factor SF and the desired variation in skill requirements $SF\alpha$, a set of possible skill factors SF_i is generated for each $i \in N^*$ using algorithms presented in Van Eynde & Vanhoucke (2022). Each activity i randomly receives a skill factor SF_i from this set. Hence, we know for each activity i how many of the $|S|$ skills it will request in the project. If $SF\alpha = 0$, the skill factor will be equal across all activities and $SF_i = SF, \ \forall i \in N^*$. If $SF\alpha > 0$, then the skill factor will vary over the activities. The higher the value of $SF\alpha$, the higher the disparity in skill factor between the activities. This $SF\alpha$ has no impact on the skill requirements if the project skill factor has an extreme value, i.e. $SF = 0.25$ or $SF = 1$. For each activity $i \in N^*$, we loop over the number of skills it requires and randomly select a new skill $j \in S$ each time. For this skill j, we then generate a random value between 1 and $r_{i,j}^{\max}$ to determine the skill requirement $r_{i,j}$. Lines 10–15 of Algorithm 2 are implemented to guarantee that for an activity i the sum of the skill requirements ($\sum_j r_{i,j}$) is not greater than the maximum activity requirement $r_i^{\max}$. To ensure this, the requirement of a random skill in set S is decremented by 1 as long as the maximum activity requirement is exceeded.

Algorithm 2 GenerateRequirements$(|N|, |S|, SF, r_i^{\max}, r_{i,j}^{\max})$.

1: *Select random* $SF\alpha \in \{0, 0.5, 1\}$
2: $SF_i, \forall i \in N^* \leftarrow$ *GenerateAlpha*$(SF \cdot |S| \cdot |N|, |N|, SF\alpha)$
3: **for** $i \in N^*$ **do**
4: $\quad$ **for** $\{1 \ldots SF_i \cdot |S|\}$ **do**
5: $\quad\quad$ **do**
6: $\quad\quad\quad$ *Select random skill* $j \in S$
7: $\quad\quad$ **while** $r_{i,j} \neq 0$
8: $\quad\quad$ $r_{i,j} \leftarrow$ *random value* $[1, r_{i,j}^{\max}]$
9: $\quad$ **end for**
10: $\quad$ **while** $\sum_j r_{i,j} > r_i^{\max}$ **do**
11: $\quad\quad$ *Select random skill* $j \in S$
12: $\quad\quad$ **if** $r_{i,j} > 1$ **then**
13: $\quad\quad\quad$ $r_{i,j} \leftarrow r_{i,j} - 1$
14: $\quad\quad$ **end if**
15: $\quad$ **end while**
16: **end for**

Table 3
Example: generation of skill availability a_j using weights w_j.

	Skill 1	Skill 2	Skill 3	Skill 4	$\sum$
$a_j^{\min}$	4	3	4	4	15
$a_j^{\max}$	12	10	15	18	55
$a_j^{\max} - a_j^{\min}$	8	7	11	14	40
w_j	0.57	0.3	0.13	0	1
w_j^a	4.56	2.1	1.43	0	8.09
$\frac{w_j^a}{\sum_j w_j^a}$	0.56	0.26	0.18	0	1
$a_j - a_j^{\min}$	7	3	2	0	12
a_j	11	6	6	4	27
SS_j	0.88	0.43	0.18	0	

`GenerateSkills()`: The skill availability a_j for each skill $j \in S$ is generated (Algorithm 3). From the project skill strength SS we are able to calculate the total number of skills in the workforce ($\sum_j \sum_k b_{j,k} = \sum_j a_j$). A set of weights $w_j \in [0, 1]$ is created based on $SS\alpha$ to introduce the desired variation in the skill availabilities based on the $\frac{\alpha}{\alpha_{\max}}$ metric (Van Eynde & Vanhoucke, 2022). The sum of these weights w_j is equal to 1 and determine how the available skills in the workforce will be divided over the skills in the project, as shown in the following example (Table 3). Obviously, the skill availability a_j is not allowed to be greater than its

Algorithm 3 GenerateSkills$(|N|, |S|, r_{i,j}, SS, SS\alpha)$.

1: $\sum_j a_j \leftarrow SS \cdot (\sum_j a_j^{\max} - \sum_j a_j^{\min}) + \sum_j a_j^{\min}$
2: $w_j, \forall j \in S \leftarrow$ *GenerateAlpha*$(1, |S|, SS\alpha)$
3: **for** $j \in S$ **do**
4: $\quad w_j^a \leftarrow w_j \cdot (a_j^{\max} - a_j^{\min})$
5: $\quad a_j \leftarrow a_j^{\min} + \frac{w_j^a}{\sum_j w_j^a} \cdot (\sum_j (a_j - a_j^{\min}))$
6: **end for**
7: **for** $j \in S$ **do**
8: $\quad$ **while** $a_j > a_j^{\max}$ **do**
9: $\quad\quad a_j \leftarrow a_j - 1$
10: $\quad\quad$ **do**
11: $\quad\quad\quad$ *Select random skill* $l \in S$
12: $\quad\quad$ **while** $j = l$ *or* $a_l \geq a_l^{\max}$
13: $\quad\quad a_l \leftarrow a_l + 1$
14: $\quad$ **end while**
15: **end for**

maximum availability $a_j^{\max}$. Therefore, we implemented a method (line 7–15) that randomly shifts the superfluous availability of skill j to other skills l.

In Table 3, we show an example of how the skill availability a_j is determined for each skill $j \in S$. In this example, we assume that the project skill strength SS is equal to 0.3 and that the values for the minimum and maximum skill availability are as shown in rows 2 and 3 of Table 3. As explained in Section 5.1, $a_j^{\max}$ and $a_j^{\min}$ can be derived from the skill requirements $r_{i,j}$. The skill requirements are not provided in this example. Next, we generate the weights w_j for each of the four skills based on the value of $SS\alpha$. The weights generated in this example coincide with an $SS\alpha$ of 0.5. If $SS\alpha = 0$, then $w_j = 0.25$, $\forall j \in S$. If $SS\alpha = 1$, then a single weight w_j will be equal to 1 and the other weights $w_l, l \in S \setminus \{j\}$, will be equal to 0. In order to take the possible range of available number of skills for each skill into account, adjusted weights w_j^a are created by multiplying the weights w_j with the possible number of skills for skill j $(a_j^{\max} - a_j^{\min})$. As follows for skill 1, $0.57 \cdot 8 = 4.56$. Since we know the value of SS, $a_j^{\max}$ and $a_j^{\min}$, we are able to determine the total skill availability $(\sum_j a_j = 0.3 \cdot 40 + 15 = 27)$. Subtracting the minimum skill availability from this total skill availability produces the number of skills that can be distributed over the skills $(\sum_j (a_j - a_j^{\min}))$. In this example, the number of skills that we can assign on top of the minimum availability is equal to 12. This value is now multiplied with the relative fraction of the adjusted weights $\frac{w_j^a}{\sum_j w_j^a}$ to establish the skill availability a_j. For skill 1, we calculate as follows, $a_1 = 0.56 \cdot 12 + 4 = 10.76 \approx 11$ (formula shown on line 5 of Algorithm 3). Since $SS\alpha = 0.5$, the skill strengths SS_j display substantial variation in this example.

`GenerateResources()`: When the skill availabilities are known, the skill distribution $b_{j,k}$ can be generated in order to create the final multi-skilled workforce (Algorithm 4). The minimum $R^{\min}$ and maximum $R^{\max}$ number of resources are calculated first as explained in Section 5.1. These values, together with the resource availability RA, can determine the total number of resources $|R|$ in the workforce. In order to introduce variability in the mastered skills of the resources, a set for the breadth $\sum_j b_{j,k}$ of all resources $k \in R$ is generated according to the skill availabilities and the $RA\alpha$ (Van Eynde & Vanhoucke, 2022). Since we know the row-sums of the skill distribution matrix $(\sum_j b_{j,k})$ from Algorithm 4 and the column-sums of the skill distribution matrix $(a_j = \sum_k b_{j,k})$ from Algorithm 3, we can generate the multi-skilled workforce. The skill distribution matrix is left-filled as shown in the left matrix of Fig. 6. In this example, we assign the first and second skill to resources 1–3. Then, the third skill is assigned to resources 1 and 4, seeing that resources 2 and 3 have already reached their number of mastered skills. Finally, skill 4 is assigned to resource 1. Afterwards, the skill distribution is scrambled, while keeping the row- and column-sums equal, by swapping skills of two random resources many times. In Fig. 6, we swap skills 1 and 3 of resources 3 and 4 without changing the row- or column-sums.

Algorithm 4 GenerateResources($|N|, |S|, r_{i,j}, RA, RA\alpha, b^{\min}, b^{\max}, a_j$).

```
R^max ← Σ_j a_j / b^min
R^min ← GetMinResources(r_{i,j}, a_j, b^min, b^max)
|R| ← RA · (R^max − R^min) + R^min
Σ_j b_{j,k}, ∀k ∈ R ← GenerateAlpha(Σ_j a_j, |R|, RAα)
for j ∈ S do
    for k ∈ R do
        b_{j,k} ← LeftfillMatrix(a_j, Σ_j b_{j,k})
    end for
end for
if LeftfillError then
    infeasibilitycounter ← infeasibilitycounter +1
    if infeasibilitycounter > 500 then
        DecrementAlpha(SSα, RAα)
        infeasibilitycounter ← 0
    end if
    go to GenerateSkills()
else
    b_{j,k} ← Scramble(b_{j,k})
end if
```

The `LeftfillMatrix()`-procedure can sometimes lead to a `LeftfillError()`. This is due to the Gale-Ryser theorem (Brualdi, 1980), which states that for a certain set of row-sums and a set column-sums a feasible matrix exists if and only if the conjugate form of the row-sums dominate the column-sums, or vice-versa. From the example in Fig. 7, we can show how in certain cases the conditions of the row-sums and column-sums can not be satisfied at the same time. On the one hand, if we conform to the row-sums and assign skill 4 to resource 2, then the skill availability of skill 2 will be 1 skill short and skill 4 will have one superfluous skill. On the other hand, assigning skill 2 to resource 3 will lead to correct skill availabilities, but the breadth of resources 2 and 3 will be incorrect. To deal with this situation and make sure that we create a feasible matrix, we start over with the `GenerateSkills()` in Algorithm 3 when a `LeftfillError()` occurs (Algorithm 1). If no feasible matrix has been generated after 500 iterations, then we decrement the value of $SS\alpha$ or $RA\alpha$ with 0.05, until a feasible matrix has been found. The chance of infeasible matrices is lower for decremented values of $SS\alpha$ and $RA\alpha$ because of the decreased disparity in the values of the row-sums and column-sums.

`FeasibilityCheck()`: The generation procedure ends with the `FeasibilityCheck()` procedure, which checks whether the generated instance is feasible. This `FeasibilityCheck()` works as a rejection method in order to find feasible instances since the problem that addresses finding a feasible multi-skilled workforce given a skill availability $\sum_{j \in S} a_j$ for a fixed number of resources $|R|$ is NP-hard. A concise version of the complexity proof for this problem is provided in Appendix B. As explained in Section 2.2, the example in Fig. 2 shows the skill requirements $r_{i,j}$ and skill distribution $b_{j,k}$ of an infeasible instance. `FeasibilityCheck()` makes use of an assignment problem to check the feasibility of the instance. Since the serial scheduling of all activities in N can lead to a feasible solution of the MSRCPSP, the feasibility of instance can be checked by confirming the feasibility of each activity separately. The mathematical formulation displayed below is solved for every activity $i \in N^*$. If this mathematical problem turns out to be infeasible for any activity, the skills and resources are generated anew. The problem specified below is similar to the fixed job scheduling problem used in Bellenguez & Néron (2007) to ensure the validity of a solution to the MSRCPSP.

$$\text{Min} \quad \sum_{j \in S} \sum_{k \in R} x_{j,k} \tag{17}$$

$$\text{s.t.} \quad x_{j,k} \le b_{j,k} \qquad \forall j \in S, k \in R \tag{18}$$

$$\sum_{k \in R} x_{j,k} = r_{i,j} \qquad \forall j \in S \tag{19}$$

$$\sum_{j \in S} x_{j,k} \le 1 \qquad k \in R \tag{20}$$

$$x_{j,k} \in \{0, 1\} \qquad \forall j \in S, k \in R \tag{21}$$

Decision variable $x_{j,k}$ is equal to 1 if resource k performs skill j. Constraints (18) ensures that the resources only perform mastered

	j=1	j=2	j=3	j=4	$\Sigma_j b_{j,k}$
k=1	1	1	1	1	4
k=2	1	1			2
k=3	1	1			2
k=4			1		1
a_j	3	3	2	1	

	j=1	j=2	j=3	j=4	$\Sigma_j b_{j,k}$
k=1	1	1	1	1	4
k=2	1	1			2
k=3		1	1		2
k=4	1				1
a_j	3	3	2	1	

Fig. 6. Left: A leftfilled $b_{j,k}$-matrix, Right: $b_{j,k}$-matrix after the scramble.

	j=1	j=2	j=3	j=4	$\Sigma_j b_{j,k}$
k=1	1	1	1	1	4
k=2	1	1	1	?	4
k=3	1	?			1
k=4	1				1
a_j	4	3	2	1	

Fig. 7. Example of the Gale-Ryser theorem.

Table 4
Parameter settings for generated data.

Parameter	Value
$\|N\|$	$\{30, 60, 90\}$
$\|S\|$	4
SP	$\{0.1, 0.3, 0.5, 0.7, 0.9\}$
$r_{i,j}^{\max}$	12
$r_i^{\max}$	4
SF	$\{0.25, 0.5, 0.75, 1\}$
SS	$\{0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1\}$
$SS\alpha$	$\{[0, 0.33], [0.33, 0.66], [0.66, 1]\}$
RA	$\{0, 0.2, 0.4, 0.6, 0.8, 1\}$
$RA\alpha$	$\{[0, 0.33], [0.33, 0.66], [0.66, 1]\}$
$b^{\min} - b^{\max}$	1–4
Instances per setting	5
Total instances	178,200

skills. The skill requirements are guaranteed in Constraints (19). Constraints (20) specify that each resource k can at most perform 1 skill. Finally, the decision variable is defined in Constraints (21).

5.3. Generator settings

Table 4 shows the values of the parameters for our generated data instances. The instances include projects with $|N| \in \{30, 60, 90\}$. For the SP, we have decided to take a wide range of values to be able to analyse the impact of the project network in multi-skilled project scheduling problems. We generated 500 networks per $SP \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ and then randomly picked from the according 500 networks for every instance. The number of skills $|S|$ in each instance is set at 4, which is the same as in the dataset of Almeida, Correia, & Saldanha-da Gama (2015). We have included the parameter value $r_{i,j}^{\max}$ which determines that the maximum number of resources that can be required by a single skill in an activity is equal to 4. Similarly, we incorporated the $r_i^{\max}$, which sets the maximum number of resources that can be required by a single activity to 12. Obviously, $r_{i,j} \le r_{i,j}^{\max}$ and $\sum_j r_{i,j} \le r_i^{\max}$. In other datasets the $r_i^{\max}$ is not specified and is left unbounded in the instances, which means that $r_i^{\max} = r_{i,j}^{\max} \cdot |S|$. However, we decided against this since an unbounded $r_i^{\max}$ substantially increases the chance that the skill requirements of a single activity would dominate the skill requirements of most other activities in the project. Consequently, this would imply that $R^{\min} \approx R^{\max}$ for low values of SS. This, in turn, means that only a very limited number of workforces can be generated across all RA values, which reduces the possibility for interesting analysis on the lower SS values. The instances in the dataset have all possible project skill factors for $|S| = 4$, meaning that $SF \in \{0.25, 0.5, 0.75, 1\}$. We have decided to generate instances with a wide range of project skill strength values, $SS \in \{0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1\}$. Furthermore, the instances have resource availabilities from 0 to 1 in steps of 0.2, $RA \in \{0, 0.2, 0.4, 0.6, 0.8, 1\}$. To clarify, for the project skill strength SS we generate instances with more values than for the resource availability RA. As mentioned in Section 5.1, in the denominator of the SS we use the total skill requirement to calculate the maximum number of skills ($a_j^{\max} = \sum_i r_{i,j}$). This choice increases the number of possible skill availabilities a_j in the project instances. Therefore, we decided to enhance the number of SS values to be able to better analyse the project skill strength and the impact of the skill availability on the objectives. For the variability parameters $SS\alpha$ and $RA\alpha$, we created three ranges from which a random value will be selected. If the $SS\alpha$ or the $RA\alpha$ is set to 'low', a value will be selected from the range [0,0.33]. A 'medium' value is randomly selected from [0.33,0.66] and a 'high' value from [0.66,1]. We decided to use ranges for the variability parameters because set values of 0, 0.5 and 1 would only lead to an analysis of extreme variability options. Additionally, it would have been hard to generate a feasible workforce each time both parameters had a value equal to 1, as we have mentioned in the previous section and in Fig. 7. Finally, the resources in the instances can be single-skilled ($b_{j,k} = b^{\min} = 1$), multi-skilled ($b_{j,k} \in \{2, \ldots, |S| - 1\}$) and completely-skilled ($b_{j,k} = b^{\max} = |S|$). Per setting, we decided to generate a total of 5 instances. As a result, the total number of generated instances is equal to 178,200.

5.4. Data overview, evaluation and validation

In this section, we will evaluate the generated instances of our data generator. In Fig. 8 we set out the parameters explained in Section 5 on two-dimensional scatterplots to analyse how the dataset covers the feature space of multi-skilled workforces in project scheduling problems. The scatterplots on the top row present the combinations of values included in the current dataset. These plots clearly show the discrete values that were decided upon and are revealed in Table 4. Obviously, it would take us too far to generate instances for all possible parameter values. To that regard we generated a representative reference set that contains values for a substantially larger amount of discrete values in order to show the range of possibilities for our data generator. The scatterplots of the reference set are presented in the bottom row of Fig. 8. The leftmost plot exhibits the skills and resources, in terms of SS and RA. The second and third scatterplot set out the skill strength SS and resource availability RA to their variabilities $SS\alpha$ and $RA\alpha$. The scatterplot on the right shows what combinations of variability are included in this dataset.

The $(SS - RA)$ scatterplots reveal that for all discrete values of project skill strength and resource availability combinations workforces have been generated in this dataset. This means that we have created a great range of workforces with different sizes and characteristics. The $(SS - SS\alpha)$ plot shows that we have a clear spread over most SS and $SS\alpha$ values except for workforces with

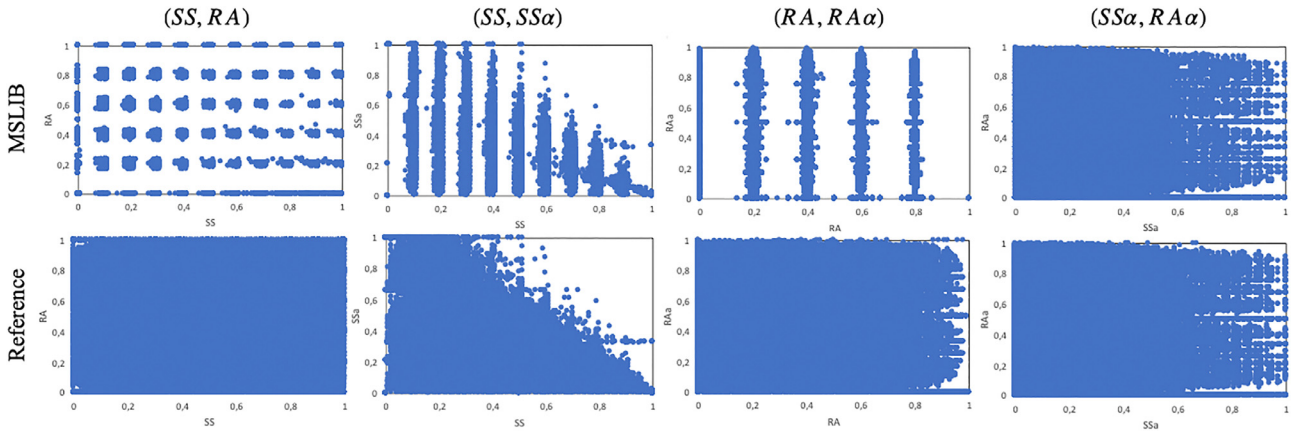


Fig. 8. Parameter scatterplots of the complete dataset and the representative scatterplots.

high SS and high $SS\alpha$ values. This is due to the fact that it is hard to create disparity (high $SS\alpha$) between the skill availabilities when there is a high number of mastered skills $\sum_j a_j$ (high SS). Moreover, the skill availability a_j per skill j is bounded by its $a_j^{\max}$ and therefore the availability is distributed over more than 1 skill which prevents great disparity between the skills. The $(RA - RA\alpha)$ scatter graph displays a slightly different parameter distribution as for the $(SS - SS\alpha)$ plot. Namely, only for RA values close or equal to 1 no workforces are featured in the dataset. The reason for this is that if $RA \simeq 1$ the number of resources is close to the number of mastered skills ($|R| = R^{\max} = \sum_j a_j$). From this it follows that most or all resources will be single-skilled, which reduces the possible disparity in the breadth of the resources. Lastly, from the $(SS\alpha - RA\alpha)$ scatterplot we can extract that for higher values of $SS\alpha$ it becomes gradually more difficult to generate instances. Since a workforce with high $SS\alpha$ possesses, in most cases, one skill that has a substantially larger availability than the other skills, the variability in the breadth of the resources will be very restricted. Nevertheless, we still generated workforces for almost all $(SS\alpha - RA\alpha)$-combinations.

In order to further evaluate our dataset, we implement the skill criticality and advanced resource criticality of Snauwaert & Vanhoucke (2021) as evaluating parameters for our new dataset. The skill criticality SC_j of a skill j is a metric for the importance of each skill in the project.

$$SC_j = \frac{\sum_i (r_{i,j} \cdot p_i)}{\sum_k b_{j,k} \cdot \sum_i p_i} \tag{22}$$

On the one hand, a skill that is required a lot in the activities, but not mastered frequently in the workforce will have a very high skill criticality. On the other hand, skills that are mastered by many resources but not required a lot will be not very critical. The advanced resource criticality RC_k^a of a resource k measures the significance of a resource by the sum of the skill criticality of its mastered skills. Therefore, a resource that masters more critical skills will be a more critical resource. The ranked skill criticality RSC_j orders the skills in terms of skill criticality such that the most critical skill gets a value of $|S|$. The formula for the advanced resource criticality is slightly modified here because this paper does not deal with the depth of the resources:

$$RC_k^a = \sum_j (RSC_j \cdot b_{j,k}) \tag{23}$$

Fig. 9 presents the average skill criticality $AvgSC$ of all skills in S and the average resource criticality $AvgRC$ of all resources in R. This graph clearly shows that our dataset includes instances with low and high average skill criticality and resources criticality. Nevertheless, it seems that no instances with high $AvgSC$ and high $AvgRC$ are covered. Since a high $AvgSC$ means that the number of required skills is close to the number of mastered skills at every point in the project, this occurs generally when the project skill factor is equal to 1. Consequently, the minimum number of resources in the workforce $R^{\min}$ will be high, which means that the resources will be predominantly single-skilled and that the average resource criticality will remain rather low. The second plot sets out the minimum and maximum skill criticality. This shows that there are instances without critical skills (bottom-left), instances with only critical skills (top-right) and instances with both low and highly critical skills (top-left). From the scatterplot on the right we can make the same conclusions. Our dataset features instances with only single-skilled resources (bottom-left), instances in which all resources are completely-skilled and highly critical (top-right) and also instances in which there is a mix in the criticality of the workforce (top-left).

In the last part of this section, we will parametrically compare the feature space of our current dataset to existing benchmark datasets in the literature. There are currently 3 datasets available in the literature for the MSRCPSP. We were able to retrieve the data used in Montoya et al. (2014) from Young, Feydy, & Schutt (2017). We implemented the data generator of Almeida et al. (2015) to generate the dataset used in Almeida, Correia, & Saldanha-da Gama (2016) and in Correia et al. (2012). Their generator utilises a modified resource strength MRS to facilitate the skill requirement and workforce generation. This MRS is fairly similar to the skill strength in our proposed research. However one value of the MRS corresponds to different values of the SS and $SS\alpha$. Myszkowski et al. (2015) proposed their iMOPSE dataset for the MSRCPSP. Their dataset has some specifics, including 1 skill requirement per activity ($\sum_j r_{i,j} = 1$) and the breadth $\sum_j b_{j,k}$ of all resources is equal to 6. As is clear from Fig. 10, our new dataset covers a larger range of the feature space in terms of resources and skills than other datasets from the literature. This may allude to the fact that these datasets did not focus on multi-skilled workforce analysis. However, our data generator also considers a greater range of project networks (determined by SP-value). Namely, we consider SP-values in the range $[0.1; 0.9]$, while Almeida et al. (2015) considers networks in the range $[0.17; 0.52]$, Montoya et al. (2014) networks in the range $[0.1; 0.38]$ and Myszkowski et al. (2015) networks in the range $[0.01; 0.2]$. In terms of multi-skilled workforce generation, we can identify from the scatterplots that the current datasets in the literature do not include instances with high resource availabilities. Therefore, multi-skilled project scheduling problems with low-skilled workforces are not represented. The dataset of Montoya et al. (2014) contains some in-

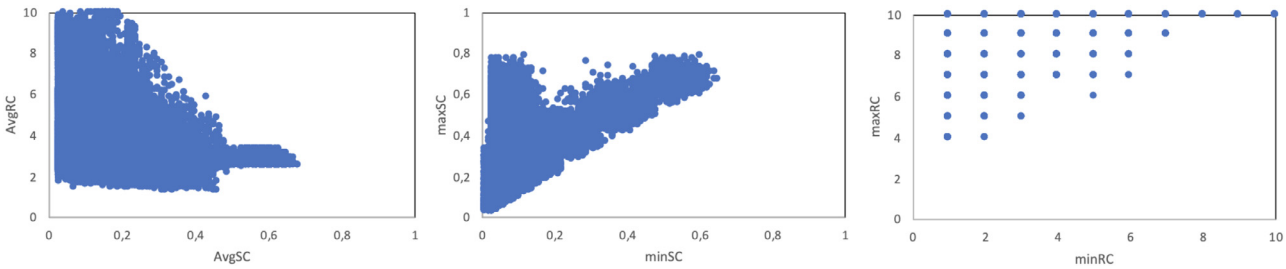


Fig. 9. Analysis of the skill and resource criticality in the dataset.

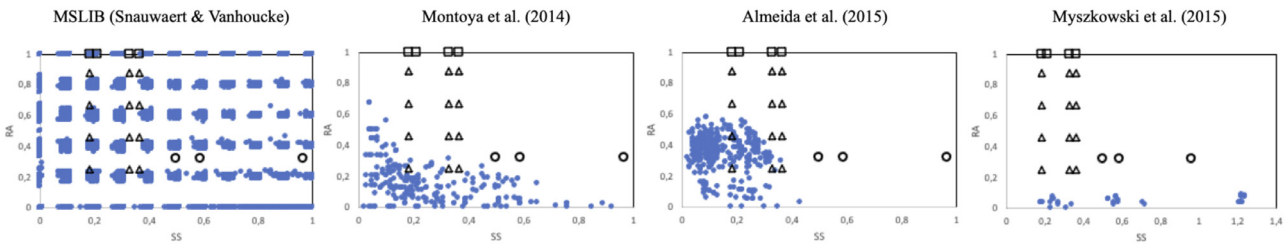


Fig. 10. Comparison of the workforce variety in literature datasets and validation with empirical instances.

stances for high skills strength values, while this is not the case for Almeida et al. (2015). Myszkowski et al. (2015) generated instances with a project skill strength higher than 1, which means that the number of mastered skills is higher than the total skill requirement in the project. Most notably is that all literature datasets only contain instances with low *SS* and low *RA* and none of them generated instances for high *SS* and *RA*. In Section 6 we analyse if these newly covered parts are as difficult as other parts of the feature space and can yield interesting insights.

In addition to artificial instances, we have also gathered 7 empirical instances for the MSRCPSP that are collected from a software company and companies that operate in the railway construction industry. These empirical instances are also shown on Fig. 10. Of these instances, 4 have a workforce with only single-skilled resources (shown as squares on the plots), while the other 3 projects were executed by a multi-skilled workforce (shown as circles on the plots). The 4 data points at the top of the scatterplots of Fig. 10 are the instances with a single-skilled workforce, i.e. single-skilled workforces have a resource availability *RA* equal to 1. The 3 empirical instances with multi-skilled workforces have a *RA* equal to 0.32. This remains the same for all 3 instances because the exact same workforce executed 3 different projects in these instances. Of course, the skill strength *SS* varies because it depends on the skill requirements of the activities in the project. These real-world applications show the relevance and importance of our new data generation procedure. Not only is it clear that the benchmark datasets do not consider single-skilled workforces in project scheduling problems, Fig. 10 also reveals that some empirical multi-skilled workforces are not considered by the datasets that are currently available.

On top of these 7 empirical instances, we discussed with one of the companies if some of the single-skilled workers could actually perform any of the other skills in order to create a multi-skilled workforce. Based on the degrees of freedom set by the company, we were able to create 12 derivative empirical instances based on 3 of the 4 single-skilled instances by removing some of the resources and adding their mastered skills to other resources (shown as triangles on Fig. 10). For example, in these software projects the jobs of the project engineer and the signalling system engineer are fairly similar and could be performed by the same resource. As such, we found that empirical instances with equal skill strength *SS* (because the number of skills remains equal) and lower resource availability *RA* (because the number of resources is reduced) are highly probable in real-life projects. If we redistribute the skills in the workforces of the empirical instances with single-skilled workforces, we can create more representative instances that can be situated in the $SS - RA$ graph. Some of these derivative instances are clearly featured in the benchmark datasets of Montoya et al. (2014) and Almeida et al. (2015), while others are left unconsidered.

In summary, we can state these empirical instances show the value of our artificial datasets and validate the data generation procedure of Section 5 and its assumptions. The complete set of empirical instances (MSLIB5), together with the instances of the 3 benchmark datasets, are available on our website https://www.projectmanagement.ugent.be/research/data.

5.5. Dataset design

In this section, we propose several subsets of the complete dataset presented in Section 5.2. The aim of these different subsets is to create multi-skilled datasets that fit the research requirements of each researcher in the field of multi-skilled project scheduling. The specifics and characteristics of the datasets will be presented along the following lines. The datasets of this section are also available on our website https://www.projectmanagement.ugent.be/research/data.

Table 5 displays an overview of the dataset design of the different sets that will be published online. In MSLIB1, we have assembled a basic set for research on the MSRCPSP. It consists of a varying set of projects with different project characteristics and of size $|N| = 30$. The instances in this set possess a large spread of multi-skilled workforces, both in terms of skills and in terms of resources. However, no considerable variability is included in the workforces. Hence, $SS\alpha$ and $RA\alpha$ are locked on the lowest setting. MSLIB2 focusses more on the project than on the workforce. This set is useful for researchers that would like to solve the MSRCPSP for a wide range of projects with a smaller but still diversified set of multi-skilled workforces. More specifically, it contains diverse projects of 30, 60 and 90 activities and workforces with a wide range of skills and resources. As in MSLIB1, the variability in the workforce is set to the lowest range. MSLIB3 is created specifically for research on multi-skilled workforce analysis. Therefore it contains a substantial set of small and large workforces with variabil-

Table 5
Parameter settings of the datasets.

Parameter	MSLIB1	MSLIB2	MSLIB3
$\|N\|$	30	{30, 60, 90}	30
SP	{0.1, 0.3, 0.5, 0.7, 0.9}	{0.1, 0.3, 0.5, 0.7, 0.9}	{0.1, 0.3, 0.5, 0.7, 0.9}
SF	{0.25, 0.5, 0.75, 1}	{0.25, 0.5, 0.75, 1}	{0.25, 0.5, 0.75, 1}
SS	{0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1}	{0.1, 0.3, 0.5, 0.7, 0.9}	{0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1}
RA	{0, 0.2, 0.4, 0.6, 0.8, 1}	{0, 0.2, 0.4, 0.6, 0.8, 1}	{0, 0.2, 0.4, 0.6, 0.8, 1}
$SS\alpha$	[0,0.33]	[0,0.33]	{[0, 0.33], [0.33, 0.66], [0.66, 1]}
$RA\alpha$	[0,0.33]	[0,0.33]	{[0, 0.33], [0.33, 0.66], [0.66, 1]}
Instances per setting	5	5	1
Total instances	6,600	9,000	11,880

Table 6
Overview of the parametric analyses in the computational experiments.

Parameter	Analysis	Difficulty analysis (CPLEX)
SP	Activity network	More serial network, difficulty ↓
SF	# of required skill types	Minimal impact
SS	# of mastered skills	# of mastered skills ↑, difficulty ↓
$SS\alpha$	Variability in skill availability	Variability ↑, difficulty ↓
RA	# of resources	# of resources ↑, difficulty ↓
$RA\alpha$	Disparity in breadth	Minimal impact

Table 7
Overview of the managerial insights.

Parameter	Analysis	Managerial insights
SS	# of mastered skills	- Parallel networks: average # of mastered skills ≤ 40% total skill requirement - Serial networks: average # of mastered skills ≤ 20% total skill requirement
RA	# of resources	- Parallel networks: average # of resources ≤ 60% mastered skills - Serial networks: # of resources has no impact on makespan
%MS	Multi-skilled % in workforce	- Multi-skilled workforces need on average 18% more skills to achieve the same makespan as a single-skilled workforce

ity in both the skill availability and the breadth of the resources. This set only considers projects of 30 activities.

6. Computational experiments

In the following section, we analyse the results from solving the MILP model of Section 2.3 for all instances of the complete dataset using CPLEX 12.6 with a time limit of 60 seconds and solving the same instances with a meta-heuristic algorithm. The computational experiments consist of two main parts. A quick overview of the computational experiments is displayed in Tables 6 and 7. First, we will analyse the impact of all parameters presented in Section 5 using the obtained solutions (see Table 6). The results will reveal the relation between the difficulty of an instance and the values of the instance parameters. Second, we provide managerial insights regarding important research questions for multi-skilled workforces (see Table 7). More specifically, the size of the workforce in terms of skills and resources is investigated as well as the multi-skilling in the workforce.

6.1. Parametric dataset analysis

In this section, we compare the results for the projects with 30, 60 and 90 activities for several values of the serial-parallel indicator *SP*, the project skill strength *SS* and the resource availability *RA* (see Table 8). In this table the solutions are classified into three main groups:

- Optimal %OPT: the percentage of instances for which CPLEX could provide an optimal solution within the time limit.
- Feasible %FNO: the percentage of instances for which a feasible but no guaranteed optimal solution was found. In this case, CPLEX provided the best found solution within the time limit, together with a lower bound.
- Infeasible 100-%OPT-%FNO: the percentage of instances for which no feasible solution was found within the time limit of 60 seconds (results not shown in Table 8).

Additionally, Table 8 displays the average gap percentage (%GAP) of the solutions from CPLEX and from a genetic algorithm (GA) based on the procedure proposed in Snauwaert & Vanhoucke (2021):

$$\%GAP = BestSol - LB^{CPLEX})/BestSol \tag{24}$$

BestSol is the lowest feasible integer solution generated by CPLEX or the GA. For the instances in %OPT this is an optimal solution, for the instances in %FNO this is the best feasible solution. LB^{CPLEX} is the lower bound determined by CPLEX by relaxing the integrality constraints, which is the default lower bound calculation in CPLEX. Note that the quality of the %GAP will depend greatly on the lower bound LB^{CPLEX} found by CPLEX. Table 8 also exhibits the average CPU time (in seconds) for the GA. Lastly, we have also added a column that explains whether the corresponding parametric values per row lead to instances that were not part of other benchmark datasets. "✓" means that this group of instances is new (and was not considered previously) and "≈" signifies that the group of instances was partially considered or nearby parametric values were generated. A blank cell means that at least one of the benchmark datasets generated instances for parameters with the according values.

SP-SS-RA Analysis: Table 8 shows that for increasing values of *SP* the number of optimal and feasible solutions for CPLEX increases. From these results, it appears that more serial projects can be solved to optimality more easily in most cases, which is in line with other research. For an $SP = 0.1$, there are groups for which the %GAP is higher than 70%, for projects with an *SP* value of 0.5 or higher this significantly decreases to averages of at most 14.8%. For the project skill strength *SS*, it is clear that the lower its value

Table 8
Comparison between instances of different sizes for various *SP*, *SS* and *RA* values.

SP	*SS*	*RA*	New	$\|N\| = 30$					$\|N\| = 60$					$\|N\| = 90$				
				CPLEX			GA		CPLEX			GA		CPLEX			GA	
				%OPT	%FNO	%Gap	%Gap	CPU	%OPT	%FNO	%Gap	%Gap	CPU	%OPT	%FNO	%Gap	%Gap	CPU
0.1	0	0	≈	0.0%	100.0%	78.9%	63.6%	12.6	0.0%	100.0%	92.8%	73.3%	45.9	0.0%	46.1%	97.6%	73.7%	110.1
		0.4		0.0%	100.0%	77.1%	67.0%	13.4	0.0%	96.1%	96.3%	71.9%	47.7	0.0%	22.2%	99.8%	72.2%	113.8
		0.6	≈	0.0%	100.0%	75.3%	66.3%	13.4	0.0%	96.7%	97.6%	70.2%	47.5	0.0%	23.9%	99.9%	72.5%	111.9
		1	✓	1.1%	98.9%	71.3%	68.8%	13.4	0.0%	79.4%	94.0%	67.9%	49.1	0.0%	11.7%	99.9%	69.8%	116.0
	0.5	0		13.9%	76.7%	61.3%	31.1%	13.4	26.7%	13.9%	68.7%	3.7%	51.2	24.4%	20.0%	70.8%	2.5%	132.1
		0.4	✓	50.6%	45.6%	32.2%	20.7%	13.5	67.8%	16.6%	30.6%	6.6%	51.2	75.6%	22.8%	24.5%	4.5%	128.7
		0.6	✓	71.1%	23.3%	20.5%	19.9%	13.4	85.6%	12.8%	14.5%	7.3%	51.0	99.4%	0.6%	0.6%	5.2%	128.2
		1	✓	93.3%	5.6%	4.1%	8.5%	13.2	100.0%	0.0%	0.0%	10.8%	51.3	100.0%	0.0%	0.0%	8.5%	123.7
	1	0	✓	34.4%	59.4%	50.1%	22.3%	14.4	51.1%	9.4%	46.3%	16.4%	49.5	48.9%	28.3%	50.6%	14.8%	126.7
		0.4	✓	98.9%	1.1%	0.7%	0.0%	14.4	100.0%	0.0%	0.0%	0.0%	49.3	100.0%	0.0%	0.0%	0.0%	118.2
		0.6	✓	100.0%	0.0%	0.0%	0.0%	14.8	100.0%	0.0%	0.0%	0.0%	49.0	77.8%	22.2%	22.2%	0.0%	126.5
		1	✓	100.0%	0.0%	0.0%	0.0%	14.8	100.0%	0.0%	0.0%	0.0%	49.2	56.7%	18.3%	43.3%	0.0%	126.3
0.5	0	0	≈	38.3%	61.7%	14.8%	8.9%	14.4	13.9%	86.1%	35.6%	15.9%	51.1	3.3%	96.7%	44.2%	20.1%	134.9
		0.4	≈	55.6%	44.4%	11.7%	12.8%	14.4	42.8%	57.2%	34.0%	18.6%	51.1	25.0%	75.0%	43.4%	22.5%	135.3
		0.6	≈	58.3%	41.7%	12.3%	14.8%	14.5	40.0%	60.0%	31.1%	17.7%	50.9	29.4%	70.6%	37.3%	21.2%	136.0
		1	✓	58.3%	41.7%	10.2%	23.5%	13.3	46.7%	53.3%	28.9%	25.3%	47.8	31.7%	68.3%	35.9%	26.6%	128.9
	0.5	0	≈	100.0%	0.0%	0.0%	0.5%	13.6	100.0%	0.0%	0.0%	0.3%	52.1	100.0%	0.0%	0.0%	0.0%	129.7
		0.4	✓	100.0%	0.0%	0.0%	0.8%	13.6	100.0%	0.0%	0.0%	0.5%	52.1	100.0%	0.0%	0.0%	0.1%	128.9
		0.6	✓	100.0%	0.0%	0.0%	0.7%	13.8	100.0%	0.0%	0.0%	0.3%	52.2	100.0%	0.0%	0.0%	0.4%	130.3
		1	✓	100.0%	0.0%	0.0%	1.6%	13.6	100.0%	0.0%	0.0%	1.0%	52.3	100.0%	0.0%	0.0%	1.1%	129.1
	1	0	✓	99.4%	0.6%	0.1%	0.0%	13.6	100.0%	0.0%	0.0%	0.0%	51.1	99.4%	0.6%	0.1%	0.0%	136.1
		0.4	✓	100.0%	0.0%	0.0%	0.0%	13.8	100.0%	0.0%	0.0%	0.0%	50.8	100.0%	0.0%	0.0%	0.0%	134.3
		0.6	✓	100.0%	0.0%	0.0%	0.0%	13.8	100.0%	0.0%	0.0%	0.0%	50.8	95.6%	3.9%	4.4%	0.0%	131.3
		1	✓	100.0%	0.0%	0.0%	0.0%	14.0	100.0%	0.0%	0.0%	0.0%	50.9	77.8%	13.9%	22.2%	0.0%	132.1
0.9	0	0	✓	99.4%	0.6%	0.1%	0.0%	12.0	78.9%	21.1%	1.3%	0.6%	46.3	70.6%	29.4%	1.5%	1.1%	133.7
		0.4	✓	98.9%	1.1%	0.1%	0.0%	12.1	85.0%	15.0%	0.9%	0.9%	47.3	80.0%	20.0%	2.2%	1.3%	131.6
		0.6	✓	98.3%	1.7%	0.2%	0.1%	11.9	87.8%	12.2%	0.7%	0.6%	47.6	76.7%	23.3%	2.5%	1.4%	131.8
		1	✓	97.2%	2.8%	0.1%	2.5%	11.8	83.9%	16.1%	0.7%	3.1%	47.6	81.1%	19.9%	0.8%	3.5%	129.7
	0.5	0	✓	100.0%	0.0%	0.0%	0.1%	13.5	100.0%	0.0%	0.0%	0.0%	48.2	100.0%	0.0%	0.0%	0.0%	131.5
		0.4	✓	100.0%	0.0%	0.0%	0.0%	14.0	100.0%	0.0%	0.0%	0.0%	49.3	100.0%	0.0%	0.0%	0.0%	131.2
		0.6	✓	100.0%	0.0%	0.0%	0.1%	13.9	100.0%	0.0%	0.0%	0.0%	49.3	100.0%	0.0%	0.0%	0.0%	131.5
		1	✓	100.0%	0.0%	0.0%	0.2%	13.9	100.0%	0.0%	0.0%	0.1%	49.6	100.0%	0.0%	0.0%	0.1%	133.8
	1	0	✓	100.0%	0.0%	0.0%	0.0%	13.7	100.0%	0.0%	0.0%	0.0%	51.3	100.0%	0.0%	0.0%	0.0%	147.9
		0.4	✓	100.0%	0.0%	0.0%	0.0%	14.0	100.0%	0.0%	0.0%	0.0%	51.2	99.4%	0.6%	0.6%	0.0%	146.1
		0.6	✓	100.0%	0.0%	0.0%	0.0%	14.1	100.0%	0.0%	0.0%	0.0%	51.3	99.4%	0.0%	0.6%	0.0%	146.3
		1	✓	100.0%	0.0%	0.0%	0.0%	14.1	100.0%	0.0%	0.0%	0.0%	51.4	74.4%	13.3%	25.5%	0.0%	146.6

the more difficult it is to find an optimal solution. Seeing that the *SS* determines the number of available skills in the workforce, we can state that there is direct relation between the number of mastered skills and the difficulty of the instance. There is a similar relation between the instance difficulty and the resource availability, albeit less convincing. Often, we find that the higher the number of resources, the lower the average %GAP is for the group of instances. A comparison of the project sizes revealed that the number of optimal and feasible solutions decreased with an increasing number of activities. Nevertheless, for some groups the number of optimal solutions increased for larger projects (e.g. $SP = 0.1$, $SS = 0.5$ and $RA = 0$), however in these cases the total number of feasible solutions still decreased. Looking at the results for the GA, we find that these are very consistent with the results for CPLEX. There are some groups for which there is a substantial difference in %GAP. More specifically, projects in the groups with an $SP = 0.1$, an $SS = 0.5$ or 1 and a low *RA* seem harder to solve for CPLEX for all project sizes. However, we can state that the hardness of solving these instances does not vary drastically between a commercial solver and a (meta-)heuristic approach.

Table 8 reveals that some parts of the feature space that were not considered by the benchmark datasets are difficult to solve (see Section 5.4). For example, instances with $SP = 0.1$, $SS = 0$ and $RA = 1$ and instances with $SS = 0.5$ and $RA \approx 0.5$ are reasonably difficult and left unconsidered. Moreover, we can see that more parallel instances with $SS = 0$ are fairly difficult to solve to optimality, while only a handful of instances for these values were previously generated. From Fig. 10 we can also see that no instances for extreme values (e.g. $SS = 0$ & $RA = 0$) are generated in the benchmark instances, which seem the most difficult to solve with both CPLEX and a meta-heuristic approach. Finally, this new dataset also generates a lot of instances for high *SP*, high *SS* and high *RA* values, but, as can be seen from the results in Table 8, these instances are rather easy to solve with either a commercial solver or a heuristic approach. However, some of these instances that are fairly easy to solve for the MSRCPSP might become difficult when the instances are used for extended variants of the MSRCPSP.

From the previous paragraph we gather that multiple instances for the MSRCPSP with varying multiskilledness and varying workforce size have not been studied, which could provoke new methodological implementations different from the current state-of-the-art approaches. An analysis of the results of the GA for this group of instances revealed that the procedure seemed to find equally satisfactory workforce assignments in the same timeframe as for instances generated from other parameter values. However, it could be interesting to see what the effect would be of pre-processing or other techniques to deal with larger workforces. Furthermore, we found that the genetic algorithm could run into problems finding feasible resource assignments for certain activities when the number of skills and resources are at extreme values close to 0. This is because, for these instances the amount of feasible resource assignments for activities are highly restricted.

SF-SS α -RA α Analysis: An analysis of the results for the other parameters presented in this research is exhibited in Table 9. The results prove that the %GAP of CPLEX increases with the project

Table 9
%Gap analysis for *SF*, *SSα* and *RAα* parameters.

SF	%OPT	%FNO	%Gap	*SSα*	%OPT	%FNO	%Gap	*RAα*	%OPT	%FNO	%Gap
0.25	92.8%	3.1%	5.0%	0	83.8%	8.4%	13.5%	0	84.8%	8.1%	12.5%
0.5	88.8%	5.1%	8.9%	0.5	85.2%	7.9%	12.3%	0.5	85.2%	8.0%	12.3%
0.75	82.8%	9.8%	14.7%	1	86.0%	7.8%	11.6%	1	85.0%	8.0%	12.5%
1	75.7%	14.0%	21.1%								

Table 10
%Gap analysis for datasets of Section 5.5.

	%OPT	%FNO	%Gap
MSLIB1	83.0%	16.3%	10.3%
MSLIB2	86.6%	8.1%	10.6%
MSLIB3	84.5%	14.9%	9.4%
MSLIB4	0.0%	93.8%	89.0%

skill factor *SF*. The more skill types that each activity requires the harder it is to solve the instance. In the middle table we can see the impact of the *SSα* on the difficulty of the instances. The impact seems rather limited which is due to the fact that quite a few instances in the dataset are solved to optimality which reduces the difference between the 3 values of *SSα*. For certain project skill strength values, the %GAP of the *SSα* can decrease with more than 20.0% between $SS\alpha = 0$ and $SS\alpha = 1$. Workforces with a greater disparity in the availability of the different skills can be solved more easily on average. The impact of the *RAα* on the %GAP is minimal. As a result, the variability in breadth of the resources has little to no impact on the difficulty of the instance. A workforce with a combination of single-skilled and completely-skilled resources will not yield a substantially different makespan from a workforce with resources of equal breadth if the number of mastered skills remains equal.

Subset Analysis: The top three rows of Table 10 report the %GAP analysis for the subsets created in Section 5.5. These results clearly show that a large part of these subsets are not so difficult to solve. Around 85% of the instances in each subset can be solved to optimality in CPLEX within 1 minute. This is due to the rapid decrease in instance difficulty for increasing *SP* and *SS* parameters which are included in these sets. Nevertheless, we believe that these subsets can still be utilised for various interesting research purposes:

- MSLIB1 is a basic set with a moderately wide range of workforces and projects. The focus lies on workforce size and the number of mastered skills in the workforce.
- MSLIB2 allows for research on larger projects with 60 and 90 activities.
- MSLIB3 is an extensive, structured set with all types of workforces in order to do detailed research on workforce comparison for projects.

However, due to these results, we decided to create one additional "hard" subset, which only focusses on the parts of the feature space that are not easily solved by CPLEX or the GA. This decision was made in order to provide a satisfactory subset for researcher that focus on creating fast and reliable algorithms for the MSRCPSP.

Hard set Analysis: We establish a "hard" set (MSLIB4) by examining the %GAP of the individual instances. The set is composed of 5,000 "hard" instances of projects with 30 activities. The specifics of this hard set are presented in Table 11. This table shows that a wide range of parameter values are considered among these 5,000 instances. More specifically, it only contains instances with *SP* values of 0.1,0.3 and 0.5. However all other parameters are included with their full range of values, which means that hard instances

Table 11
Parameter settings for the hard MSLIB4.

Parameter	MSLIB4 (hard)
$\|N\|$	30
SP	{0.1, 0.3, 0.5}
SF	{0.25, 0.5, 0.75, 1}
SS	{0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1}
RA	{0, 0.2, 0.4, 0.6, 0.8, 1}
SSα	{[0, 0.33], [0.33, 0.66], [0.66, 1]}
RAα	{[0, 0.33], [0.33, 0.66], [0.66, 1]}
Total instances	5,000

can be generated for a wide spread of *SS*, *RA*, *SSα* and *RAα* values. Of course, some particular values are more frequent than others in this hard subset. The bottom row of Table 10 shows that not a single instance of this set can be solved to optimality by CPLEX in 1 minute. For 93.8% of the instances a feasible, but no guaranteed optimal solution was found. The resulting total average %GAP of all instances in the subset is equal to 89.0%, which is a great increase compared to the first 3 MSLIB datasets.

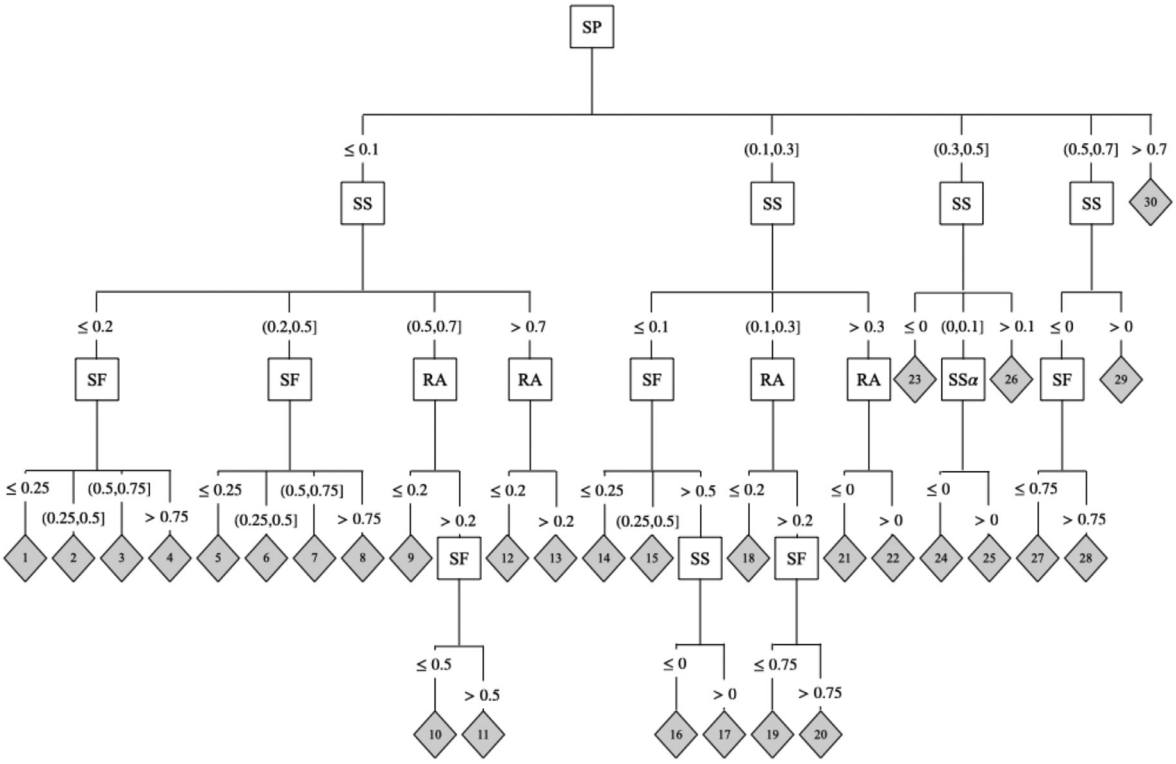
- MSLIB4 is a set composed of the hardest instances with projects of 30 activities. The purpose of this set is to test quality of solution approaches from the literature.

6.2. Experimental analysis

6.2.1. Predictive power of parameters on instance hardness

In the previous section, we have presented the results of a genetic algorithm and an exact solution procedure with CPLEX on the generated instances. In this section, we will utilize the same results to analyse the predictive power of the parameters on the hardness of the generated instances. This analysis allows us to investigate the influence of our workforce parameters on instance hardness. In order to quantify the discriminatory power of our parameters, we propose a regression tree based on the CHAID algorithm (Kass, 1980) that produces a concise graph that can facilitate a priori estimations on the hardness of an instance based on its parameter values. We use the %GAP of CPLEX as the dependent variable to express the hardness of the instances. The independent variables are the parameters of Section 5 and the network parameter *SP*, with all parameter settings shown in Table 4. The dataset on which this analysis is performed consists of all generated instances with the complete parameter ranges that consist of 30 activities, resulting in a total of 59,400 instances. We perform a split-sample validation by randomly creating a training set that consists of 5% of the complete dataset, while the test set is made up of the other 95%. Although a 5% training set is fairly small, taking into account the substantial size of the dataset (59,400 instances) and factoring in the number of classes and the input features of the independent variables the training set of 2,970 instances yields satisfying results. We have to note that this CHAID regression tree only shows the hardness of instances that are being solved by CPLEX and other solution approaches might show different results.

The CHAID algorithm starts off by splitting the total set of instances into two or more subgroups using one of its independent variables. An *F*-test is performed in order to determine the split-



1. (32.44%,22.39)	7. (53.16%,39.97)	13. (1.59%,11.63)	19. (0.19%,1.48)	25. (0.16%,0.74)
2. (61.88%,17.39)	8. (76.84%,36.99)	14. (5.02%,12.21)	20. (5.65%,12.78)	26. (0.28%,3.47)
3. (92.10%,14.51)	9. (46.61%,44.45)	15. (22.56%,17.39)	21. (5.11%,19.21)	27. (0.22%,1.20)
4. (99.84%,0.44)	10. (0.40%,2.38)	16. (62.27%,17.15)	22. (0.21%,3.65)	28. (9.60%,9.83)
5. (2.29%,7.65)	11. (18.66%,37.38)	17. (41.78%,31.46)	23. (12.01%,16.35)	29. (0.05%,1.22)
6. (22.30%,26.10)	12. (30.77%,41.13)	18. (11.63%,21.93)	24. (5.07%,10.29)	30. (0.00%,0.00)

Legend: End node. (average %GAP, standard deviation)

Fig. 11. CHAID regression tree for the MSRCPSP.

ting (independent) variable as well as the splitting value that will be most effective in reducing the variation in the groups of instances. The splitting variables and the split values with the lowest *p*-value will comprise the split values of the CHAID regression. The main reason for using an *F*-test is that our dependent variable %GAP is continuous. For this *F*-test, we use a significance level $\alpha = 0.05$. The tested hypothesis is the null hypothesis that tests whether the model without independent variables fits the data as well as the model with the parameters. Afterwards, the same process is performed for the created subgroups until no more significant subgroups can be created. An end node in the CHAID regression tree is reached when no *p*-value is lower than 0.05, which means that no significant difference in variation is found. Other stopping criteria are that the minimum number of instances per parent node in the CHAID regression tree is set at 50, per child node it is set at 15 and the maximum depth is equal to 4.

Fig. 11 displays the CHAID regression tree for instances of the MSRCPSP with a total of 47 nodes, 30 of which are end nodes. Each square node in the tree represents a split value of the mentioned independent variable, each diamond node represents an end node. In the legend below the regression tree, we provide the average %GAP and the standard deviation of the subgroup of instances that coincide with the end node. In the CHAID regression tree of Fig. 11 the *SP*, *SF*, *SS*, *SSα* and *RA* are included independent variables. Of these the *SP*, *SF* and *SS* are the highest influencing independent parameters.

- **The serial-parallel network indicator value** *SP* has a high impact on the hardness of the instances, which is why it is utilised as the first splitting variable. From the legend below Fig. 11, this predictive analysis shows that instances with lower *SP* values are more likely to be harder. It is confirmed in previous literature that the structure of the network is known to influence the hardness of project scheduling instances. For more information on the impact of *SP* on the hardness of project scheduling instances, we refer the reader to Vanhoucke et al. (2008).
- **The project skill factor** *SF* studies the impact of the size of the skill requirements of the instance hardness. The CHAID regression tree predicts that instances with higher *SF* values will be harder to solve, as can be seen by the increasing %GAP in end nodes 1 to 4 and end nodes 5 to 8.
- **The project skill strength** *SS* has substantial predictive power for the hardness of MSRCPSP instances. A lower number of available skills in the instance (i.e. a low *SS*) seems to correlate with high %GAP values.
- **The skill strength variability** *SSα* only appears once in the CHAID regression tree. Therefore, we can state the disparity in

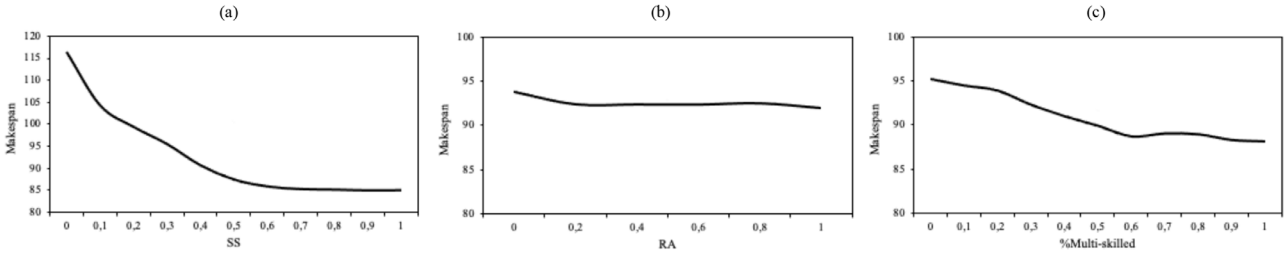


Fig. 12. Impact of parameters on project makespan: (a) *SS*, (b) *RA*, (c) % of multi-skilled resources in the workforce.

skill availability is not a good measurement to predict to hardness of instances.

- The impact of **the resource availability** *RA* is limited because its range is highly dependent on the *SS*. If the *SS* is low, then the number of available skills will be low. In that case, the impact of the *RA* is minimal since the number of possible workforce sizes will be small and their range will be narrow. If the *SS* is high, then there will be a high number of skills and the *RA* can have a larger impact on the instance hardness. In the CHAID regression tree, we can see that the *SF* has an impact on the subgroups with low *SS* while *RA* has an impact on the subgroups with high *SS*.
- **The resource availability variability** $RA\alpha$ is not included in the CHAID regression tree, which means that the multiskilledness of the workforce has little to no impact on the hardness of the instance.

6.2.2. Impact of parameters on makespan

Since our dataset spans the full range of complexity, this section presents an illustrative experimental analysis that shows the impact of the characteristics and the size of a multi-skilled workforce on the makespan of the project. Similar to the CHAID regression, the analysis in this section is performed on all instances of 30 activities (59,400 instances), however in this case the results of the GA are studied. Below, we use a statistical regression to analyse the impact of three workforce parameters: (1) project skill strength *SS*, (2) resource availability *RA*, (3) multi-skilling in the workforce. For each parameter, we test the null hypothesis, i.e. that the parameter has no impact on the makespan. The significance level for this regression is set $\alpha = 0.05$. If the *p*-value $< \alpha$, we can reject the null hypothesis and are sure that the parameter has a significant impact on the makespan.

Number of Mastered Skills: The first analysis deals with the amount of mastered skills that should be part of the total workforce depending on the type of project. To analyse this, we disregard the resources and only study the total number of mastered skills. From graph (a) of Fig. 12, we found that from a certain number of mastered skills the makespan does not improve any longer, while having additional available skills will definitely increase the cost. The results indicate that the amount of skills should be lower than 60% of the total skill requirement as can be seen in this graph. All types of workforces with a number of mastered skills lower than this 60% can be optimal depending on a particular deadline, budget or other influencing factors. We performed a statistical regression that assured that this downward trend for increasing *SS* is indeed significant.

Workforce Size: Next, we study the size of the workforce using the resource availability *RA*. More concretely, we examine how big a workforce should be in number of resources. The results in graph (b) of Fig. 12 display that the number of resources in a multi-skilled workforce has a lower impact on the final makespan than the number of mastered skills in a workforce. Statistical regression showed that there is no significant relation between the number of resources and the final makespan of the project. Hence, the makespan is only affected by the number of skills, but how they are divided over the resources has no impact. As long as the number of mastered skills remains the same, the workforce can be both single-skilled or completely-skilled without influencing the project makespan.

Multi-skilling: Finally, we investigated what the impact is of the number of multi-skilled resources in the workforce on the makespan. To calculate the percentage of multi-skilled resources in a workforce, we use the definition of multi-skilled resources as presented in Section 2, namely, a resource is multi-skilled if it masters 2 or more skills. The results reveal that the lower the % of multi-skilled workers is, the higher the makespan is (see graph (c) of Fig. 12). Therefore, we can state that multi-skilled workforces have a positive impact on the makespan in comparison to single-skilled workforces. These results are also confirmed as being significant through statistical regression.

7. Conclusion

This paper studies the MSRCPSP, which deals with the scheduling of project activities and the assignment of the multi-skilled resources. This problem extends the RCPSP by introducing skill requirements and multi-skilled resources constraints. For the purpose of creating a good baseline for future research in multi-skilled projects, this paper contributed to the multi-skilled project scheduling literature in several ways. First, this paper proposes a classification scheme that extends the work of Herroelen et al. (1999) for multi-skilled project scheduling research. Second, a data generator procedure is presented with multiple new multi-skilled parameters. Several datasets with varying purposes, which are available to the public, have been generated using this procedure. This paper shows that these datasets incorporate parts of the feature space that were previously left unconsidered in the literature. Third, 7 empirical projects and several derivative instances are collected and are also made available. These empirical projects are used to guarantee the validity and relevance of our data generator. Last, computational experiments and managerial insights have been analysed and explained that contribute to the understanding of the desired multi-skilled workforce characteristics in projects.

The future research perspectives are fourfold. A first research perspective is to develop new exact and heuristic algorithms for the MSRCPSP and its extensions. These new solution approaches can be tested and compared on the dataset presented in this paper seeing that it includes new parts of the feature space that were previously left unconsidered in benchmark datasets. Moreover, the new instances can be extended in a straightforward manner to deal with extensions of the MSRCPSP with hierarchical skills such as the ones explained in the field α_6 of our classification. Another interesting research direction is to verify whether the current state-of-the-art approaches are as qualitative for the new instances in this dataset as they are for the previously available instances. Next, a stochastic approach to scheduling problems has been researched for the RCPSP, but the stochastic version of the MSRCPSP has not been studied frequently. Since this paper purely looks at the deter-

ministic side of the MSRCPSP, a clear future research perspective is to study the impact of stochasticity on the MSRCPSP. Finally, based on the framework in this paper, new empirical data instances and empirical validation can be done in the future for project scheduling problems with multi-skilled resources.

Acknowledgments

We acknowledge the support provided by the Nationale Bank van BelgiŁ (NBB) and the Bijzonder Onderzoeksfonds (BOF) for the project, under contract number BOF12GOA021.

The computational resources (Stevin Supercomputer Infrastructure) and services used in this work were provided by the VSC (Flemish Supercomputer Center), funded by Ghent University, FWO and the Flemish Government - department EWI.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.ejor.2022.05.049.

References

Afshar-Nadjafi, B. (2020). Multi-skilling in scheduling problems: A review on models, methods and applications. *Computers & Industrial Engineering*, 107004.

Almeida, B., Correia, I., & Saldanha-da Gama, F. (2015). An instance generator for the multi-skill resource-constrained project scheduling problem. *Faculdade de Ciências da Universidade de Lisboa-Centro de Matemática, Aplicações Fundamentais e Investigação Operacional*.

Almeida, B. F., Correia, I., & Saldanha-da Gama, F. (2016). Priority-based heuristics for the multi-skill resource constrained project scheduling problem. *Expert Systems with Applications, 57*, 91–103.

Almeida, B. F., Correia, I., & Saldanha-da Gama, F. (2019). Modeling frameworks for the multi-skill resource-constrained project scheduling problem: A theoretical and empirical comparison. *International Transactions in Operational Research, 26*(3), 946–967.

Alvanchi, A., Lee, S., & Abou Rizk, S. M. (2012). Dynamics of workforce skill evolution in construction projects. *Canadian Journal of Civil Engineering, 39*(9), 1005–1017.

Avramidis, A. N., Chan, W., Gendreau, M., L'ecuyer, P., & Pisacane, O. (2010). Optimizing daily agent scheduling in a multiskill call center. *European Journal of Operational Research, 200*(3), 822–832.

Bellenguez, O., & Néron, E. (2004). Lower bounds for the multi-skill project scheduling problem with hierarchical levels of skills. In *International conference on the practice and theory of automated timetabling* (pp. 229–243). Springer.

Bellenguez, O., & Néron, E. (2007). A branch-and-bound method for solving multi--skill project scheduling problem. *RAIRO-Operations Research, 41*(2), 155–170.

Blazewicz, J., Lenstra, J. K., & Kan, A. R. (1983). Scheduling subject to resource constraints: classification and complexity. *Discrete Applied Mathematics, 5*(1), 11–24.

Brualdi, R. A. (1980). Matrices of zeros and ones with fixed row and column sum vectors. *Linear Algebra and its Applications, 33*, 159–231.

Cai, X., & Li, K. N. (2000). A genetic algorithm for scheduling staff of mixed skills under multi-criteria. *European Journal of Operational Research, 125*(2), 359–369.

Certa, A., Enea, M., Galante, G., & Manuela La Fata, C. (2009). Multi-objective human resources allocation in r&d projects planning. *International Journal of Production Research, 47*(13), 3503–3523.

Chen, R., Liang, C., Gu, D., & Leung, J. Y. (2017). A multi-objective model for multi-project scheduling and multi-skilled staff assignment for it product development considering competency evolution. *International Journal of Production Research, 55*(21), 6207–6234.

Chen, R., Liang, C., Gu, D., & Zhao, H. (2020). A competence-time-quality scheduling model of multi-skilled staff for it project portfolio. *Computers & Industrial Engineering, 139*, 106183.

Chen, W.-N., & Zhang, J. (2012). Ant colony optimization for software project scheduling and staffing with an event-based scheduler. *IEEE Transactions on Software Engineering, 39*(1), 1–17.

Correia, I., Lampreia-Lourenço, L., & Saldanha-da Gama, F. (2012). Project scheduling with flexible resources: formulation and inequalities. *OR Spectrum, 34*(3), 635–663.

Dang Quoc, H., Nguyen Doan, C., Xiong, N., et al., (2020). Effective evolutionary algorithm for solving the real-resource-constrained scheduling problem. *Journal of Advanced Transportation, 2020*.

De Bruecker, P., Beliën, J., Van den Bergh, J., & Demeulemeester, E. (2018). A three--stage mixed integer programming approach for optimizing the skill mix and training schedules for aircraft maintenance. *European Journal of Operational Research, 267*(2), 439–452.

De Bruecker, P., Van den Bergh, J., Beliën, J., & Demeulemeester, E. (2015). Workforce planning incorporating skills: State of the art. *European Journal of Operational Research, 243*(1), 1–16.

Drezet, L.-E., & Billaut, J. C. (2008). A project scheduling problem with labour constraints and time-dependent activities requirements. *International Journal of Production Economics, 112*(1), 217–225.

Felberbauer, T., Gutjahr, W. J., & Doerner, K. F. (2019). Stochastic project management: Multiple projects with multi-skilled human resources. *Journal of Scheduling, 22*(3), 271–288.

Fırat, M., & Hurkens, C. A. (2012). An improved MIP-based approach for a multi-skill workforce scheduling problem. *Journal of Scheduling, 15*(3), 363–380.

Hartmann, S., & Briskorn, D. (2010). A survey of variants and extensions of the resource-constrained project scheduling problem. *European Journal of Operational Research, 207*(1), 1–14.

Hartmann, S., & Briskorn, D. (2021). An updated survey of variants and extensions of the resource-constrained project scheduling problem. In *European journal of operational research*.

Hegazy, T., Shabeeb, A. K., Elbeltagi, E., & Cheema, T. (2000). Algorithm for scheduling with multiskilled constrained resources. *Journal of Construction Engineering and Management, 126*(6), 414–421.

Hematian, M., Seyyed Esfahani, M. M., Mahdavi, I., Mahdavi-Amiri, N., & Rezaeian, J. (2020). A multiobjective integrated multiproject scheduling and multiskilled workforce assignment model considering learning effect under uncertainty. *Computational Intelligence, 36*(1), 276–296.

Herroelen, W., Demeulemeester, E., & De Reyck, B. (1999). A classification scheme for project scheduling. In *Project scheduling* (pp. 1–26). Springer.

Herroelen, W., & Leus, R. (2005). Project scheduling under uncertainty: Survey and research potentials. *European Journal of Operational Research, 165*(2), 289–306.

Javanmard, S., Afshar-Nadjafi, B., & Niaki, S. T. A. (2017). Preemptive multi-skilled resource investment project scheduling problem: Mathematical modelling and solution approaches. *Computers & Chemical Engineering, 96*, 55–68.

Kass, G. V. (1980). An exploratory technique for investigating large quantities of categorical data. *Journal of the Royal Statistical Society: Series C (Applied Statistics), 29*(2), 119–127.

Kolisch, R., & Heimerl, C. (2012). An efficient metaheuristic for integrated scheduling and staffing it projects based on a generalized minimum cost flow network. *Naval Research Logistics, 59*(2), 111–127.

Kolisch, R., Sprecher, A., & Drexl, A. (1995). Characterization and generation of a general class of resource-constrained project scheduling problems. *Management Science, 41*(10), 1693–1703.

Labro, E., & Vanhoucke, M. (2008). Diversity in resource consumption patterns and robustness of costing systems to errors. *Management Science, 54*(10), 1715–1730.

Liu, S.-S., & Wang, C. J. (2012). Optimizing linear project scheduling with multi--skilled crews. *Automation in Construction, 24*, 16–23.

Maghsoudlou, H., Afshar-Nadjafi, B., & Niaki, S. T. A. (2016). A multi-objective invasive weeds optimization algorithm for solving multi-skill multi-mode resource constrained project scheduling problem. *Computers & Chemical Engineering, 88*, 157–169.

Maghsoudlou, H., Afshar-Nadjafi, B., & Niaki, S. T. A. (2019). Preemptive multi-skilled resource constrained project scheduling problem with hard/soft interval due dates. *RAIRO-Operations Research, 53*(5), 1877–1898.

Maghsoudlou, H., Afshar-Nadjafi, B., & Niaki, S. T. A. (2017). Multi-skilled project scheduling with level-dependent rework risk; three multi-objective mechanisms based on cuckoo search. *Applied Soft Computing, 54*, 46–61.

Montoya, C., Bellenguez-Morineau, O., Pinson, E., & Rivreau, D. (2014). Branch-and-price approach for the multi-skill project scheduling problem. *Optimization Letters, 8*(5), 1721–1734.

Myszkowski, P., Skowroński, M., Olech, L., & Oślizło, K. (2015). Hybrid ant colony optimization in solving multi-skill resource-constrained project scheduling proble. *Soft Computing, 19*(12), 3599–3619.

Najafzad, H., Davari-Ardakani, H., & Nemati-Lafmejani, R. (2019). Multi-skill project scheduling problem under time-of-use electricity tariffs and shift differential payments. *Energy, 168*, 619–636.

Néron, E. (2002). Lower bounds for the multi-skill project scheduling problem. In *Proceeding of the eighth international workshop on project management and scheduling* (pp. 274–277).

Othman, S. B., Hammadi, S., & Quilliot, A. (2015). Multi-objective evolutionary for multi-skill health care tasks scheduling. *IFAC-PapersOnLine, 48*(3), 704–709.

Othman, S. B., Zgaya, H., Hammadi, S., Quilliot, A., Martinot, A., & Renard, J. M. (2016). Agents endowed with uncertainty management behaviors to solve a multiskill healthcare task scheduling. *Journal of Biomedical Informatics, 64*, 25–43.

Polo-Mejía, O., Artigues, C., Lopez, P., & Basini, V. (2019). Mixed-integer/linear and constraint programming approaches for activity scheduling in a nuclear research facility. *International Journal of Production Research*, 1–18.

Snauwaert, J., & Vanhoucke, M. (2021). A new algorithm for resource-constrained project scheduling with breadth and depth of skills. *European Journal of Operational Research, 292*(1), 43–59.

Valls, V., Pérez, A., & Quintanilla, S. (2009). Skilled workforce scheduling in service centres. *European Journal of Operational Research, 193*(3), 791–804.

Van Eynde, R., & Vanhoucke, M. (2022). New summary measures and datasets for the multi-project scheduling problem. *European Journal of Operational Research, 299*(3), 853–868.

Vanhoucke, M., Coelho, J., Debels, D., Maenhout, B., & Tavares, L. V. (2008). An evaluation of the adequacy of project network generators with systematically sampled networks. *European Journal of Operational Research, 187*(2), 511–524.

Yannibelli, V., & Amandi, A. (2011). A knowledge-based evolutionary assistant to software development project scheduling. *Expert Systems with Applications, 38*(7), 8403–8413.

Yannibelli, V., & Amandi, A. (2013). Hybridizing a multi-objective simulated annealing algorithm with a multi-objective evolutionary algorithm to solve a multi--objective project scheduling problem. *Expert Systems with Applications, 40*(7), 2421–2434.

Young, K. D., Feydy, T., & Schutt, A. (2017). Constraint programming applied to the multi-skill project scheduling problem. In *International conference on principles and practice of constraint programming* (pp. 308–317). Springer.

Zabihi, S., Kahag, M. R., Maghsoudlou, H., & Afshar-Nadjafi, B. (2019). Multi-objective teaching-learning-based meta-heuristic algorithms to solve multi-skilled project scheduling problem. *Computers & Industrial Engineering, 136*, 195–211.

Zhu, L., Lin, J., Li, Y.-Y., & Wang, Z. J. (2021). A decomposition-based multi-objective genetic programming hyper-heuristic approach for the multi-skill resource constrained project scheduling problem. In *Knowledge-based systems* (p. 107099).